

Received 7 April 2025, accepted 5 May 2025, date of publication 20 May 2025, date of current version 6 June 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3571737

RESEARCH ARTICLE

Performance-Based Scheduling in Distributed Mixed Criticality Systems

AMJAD ALI¹, SAUD WASLY², ASAD MASOOD KHATTAK³, (Member, IEEE), IHSAN ALI¹, SHAHID IQBAL¹, AND BASHIR HAYAT⁴

¹Department of Computer and Software Technology, University of Swat, Swat, Saidu Sharif, Khyber Pakhtunkhwa 19130, Pakistan

²Electrical and Computer Engineering Department, King Abdulaziz University, Jeddah 21589, Saudi Arabia

³College of Technological Innovation, Zayed University, Dubai, United Arab Emirates

⁴School of Computer Science and IT, Institute of Management Sciences Peshawar, Peshawar 25000, Pakistan

Corresponding author: Bashir Hayat (bashir.hayat@imsiences.edu.pk)

This work was supported in part by the Deanship of Scientific Research (DSR) at King Abdulaziz University, Jeddah, Saudi Arabia, under Grant GPIP-2038-135-2024; and in part by Zayed University Research Incentive Fund under Grant 23078.

ABSTRACT With a focus on computationally intensive, distributed, and parallel workloads, scheduling in mixed-criticality distributed systems presents significant challenges due to shared memory and resources, as well as the diverse demands of tasks. The system's efficiency is heavily dependent on the overall scheduling duration (make span), while individual task deadlines impose strict timing constraints. When the tasks need to simultaneously access the shared memory, then these tasks interfere the execution of one another. For managing the scheduling of interfering tasks in distributed mixed-criticality systems, a novel Interference-Aware Partitioning Fixed Priority (IAP-FP) approach is proposed, which effectively handles task partitioning among cores while considering interference, ensuring better performance and adherence to critical deadlines. This method reduces task waiting times, which minimizes the scheduling duration and improving the schedulability of the task set. The novel proposed approach is compared with Mixed-Criticality Multicore Compositional Earliest Deadline First (MMC-EDF), Global and Mixed Criticality Partitioning (MC-Partitioning) approaches to show the efficiency. The proposed approach schedules 75% tasks while the MMC-EDF, Global and MC-Partitioning approaches schedules 65%, 0% and 18% tasks respectively for target utilization $U=0.8$. As the utilization of mixed criticality (MC) workload increases, the schedulability of MC task sets decreases, but still the proposed approach performs better than the MMC-EDF, Global and MC-partitioning approaches.

INDEX TERMS Mixed criticality systems, real-time systems, distributed mixed criticality systems, fixed priority, interference aware.

I. INTRODUCTION

Mixed-criticality systems are computing or embedded platforms built to manage and run multiple tasks, applications, or components that differ in their safety and criticality levels, all within a single system. These systems are fundamental in sectors like aerospace, automotive, healthcare, and industrial automation, where such diverse demands are common. The concept of mixed-criticality highlights the ability of these systems to process both safety-critical and less critical tasks simultaneously, ensuring seamless operation [1].

The associate editor coordinating the review of this manuscript and approving it for publication was Shih-Wei Lin¹.

Since Vestal's seminal work in 2007, mixed-criticality systems have been a focal point of extensive research. Key areas of study include system modeling, analytical methods, scheduling algorithms, shared resource management, and broader system challenges. Such systems consolidate components with different levels of criticality onto a single hardware platform [2]. These components typically consist of tasks that share resources, requiring efficient scheduling to ensure that no task misses its deadline, especially high-criticality tasks. Failure to schedule high-criticality tasks appropriately can lead to severe consequences, such as financial losses, environmental damage, equipment destruction, or even loss of life. In contrast, missing deadlines for low-criticality tasks may

result in degraded system performance without catastrophic outcomes. A practical example is the safety design of a wind turbine control system. The system's high-criticality tasks involve maintaining the turbine's operational safety within its design limits. Simultaneously, the same system also manages low-criticality tasks, such as controlling video surveillance cameras or facilitating communication with maintenance personnel. Here, the turbine control tasks are crucial for safety, whereas the tasks related to video surveillance or operator communication have lower criticality [3].

Mixed-criticality systems are increasingly being implemented on multiprocessor platforms. These platforms accommodate a growing number of functionalities with varying levels of criticality, all operating on a shared hardware framework, resulting in complex system designs [4]. The scheduling techniques for such multiprocessor systems are generally divided into three key categories: global scheduling, partitioned scheduling, and cluster-based scheduling algorithms. Global scheduling involves assigning tasks to multiple processors within a system. In mixed-criticality real-time systems, global scheduling algorithms manage the priority assignment and schedulability of tasks. Researchers have been exploring the use of global, fixed-priority algorithms for multiprocessor mixed-criticality systems. The two main challenges in this approach are priority assignment and schedulability testing, and various algorithms have been developed to address these issues [5]. In global scheduling, tasks that arrive are initially placed in a queue. The tasks are then selected based on their priority, with the highest priority tasks being executed by different processors. This approach allows for task preemption and migration across processors [4].

The second type of scheduling is Partitioning Scheduling Algorithms, where tasks are divided into separate partitions, with each partition assigned to a single processor. In this approach, the multiprocessing system is essentially converted into several uniprocessor systems because task migration between processors is not allowed [6]. Tasks within each partition are scheduled separately from one another. This method enhances isolation, resource allocation, and overall predictability, while also improving fault tolerance. Compared to global scheduling, partitioned scheduling is often more effective for MC systems, as it utilizes more specific schedulability analysis techniques. These methods are becoming more widely used in industrial standards for managing the scheduling of multicore real-time systems.

The third category of scheduling algorithms for multiprocessor systems is cluster-based scheduling. In some cases, task sets that cannot be scheduled by global scheduling can be successfully scheduled using partitioning algorithms, and vice versa. The need for a hybrid approach that leverages the advantages of both global and partitioned scheduling while addressing their limitations has become increasingly evident [7]. Cluster scheduling offers a practical solution by grouping cores into clusters and managing tasks within those clusters. Each cluster is overseen by a global scheduler

to optimize task allocation, and task assignment to clusters typically relies on a worst-fit heuristic. To evaluate the schedulability of tasks within a cluster, fixed-priority response time analysis, grounded in Audsley's theoretical framework, is used. If a high-priority critical task cannot complete within its allotted time in low mode, the system transitions to high-criticality mode.

Cluster scheduling is flexible and can be adapted for either partitioned or global scheduling, depending on how the cores and clusters are configured. This approach has proven valuable as it enables a higher percentage of task sets to be successfully scheduled compared to other methods [8]. In the context of real-time distributed systems, cluster scheduling enhances performance while reducing scheduling overhead. By grouping processing nodes into clusters and managing tasks within these clusters, it simplifies the complexities of real-time scheduling. Whether implemented as a global or partitioned scheduler depends on the specific cluster and core setup. Its key benefits include better system utilization, lower migration and preemption overhead, and improved load balancing, all of which contribute to superior performance across distributed nodes [9].

As embedded systems continue to evolve, they are transitioning into distributed systems with heterogeneous architectures, such as the one found in automotive electronic systems. These systems consist of various Electronic Control Units (ECUs) spread across multiple communication buses, all interconnected through a central gateway [10]. The resources in these systems are linked via high-speed networks, forming a new platform for distributed computing. This setup enables the execution of computationally demanding and parallel applications. For such a distributed system to function effectively, it requires support both at compile time and run time for the execution of applications. To achieve optimal performance, a well-designed scheduling strategy is essential to efficiently allocate tasks across available resources. One of the main challenges in task scheduling involves assigning tasks to the appropriate processors and determining their execution priorities.

The distributed systems encompass both scheduling and communication challenges, this work specifically focuses on optimizing task-to-core assignment to minimize interference of MC tasks to ensure timing guarantees in distributed real-time systems for better schedulability. This study primarily aims to develop an effective method for assigning and organizing mixed criticality tasks on cores within a distributed system. The focus is on distributed tasks that could be executed in systems like smart health management, smart cities, and smart homes, which depend on the Internet of Things. These elements are crucial for the future of society and function within the framework of mixed criticality distributed systems. The research has the potential to pave the way for future investigations, fostering the creation of more sophisticated approaches and robust systems for scheduling tasks in distributed mixed-criticality environments.

Contributions

- Suggests a novel task-to-core allocation strategy for distributed mixed-critical systems to reduce interference and resolves conflicts among tasks competing for common resources.
- Presenting an interference-aware scheduling framework that minimizes waiting times and provide timing guarantees, thus improving schedulability in distributed systems.

II. RELATED WORK

The mixed criticality systems are being adopted in most of the modern day embedded systems like modern vehicles and aeronautics for meeting the very high nonfunctional requirements such as weight, systems budget, less space, less power requirement and generation. In fact, mixed criticality problems have been addressed in avionics domains and automotive industrial as well [10]. One of the biggest challenges is how to appropriately share resources across more than one processor. These include setting how to allocate processor time, memory, and tasks. It is difficult to know whether or not a particular set of tasks can be scheduled within the constraints. This includes evaluating deadlines on tasks, priorities among them, and resources needed to accomplish each task effectively. In summary, and in addition to mixed-criticality, the more appropriate scheduling policy and its allocation resource for multi-processor, schedulability analysis with preemption and synchronization presents new challenges.

A Global Flexible Time-Triggered Scheduling Algorithm (FTTS) is adopted as a scheduling algorithm for periodic tasks with different criticality in shared multi-core systems. The algorithm is designed to execute tasks of equal criticality together in order to meet their respective deadlines. The FTTS functions in a cycle whose length is equal to the least common multiplier of each task periods along with being subdivided into frames, which could have different sizes [4]. The Global Fixed Priority Algorithm was used by another researcher [11] to schedule both criticality tasks over multiple cores. Which deal with scheduling of criticality sporadic tasks on multi-processors using response time analysis.

→ To solve the problems of scalability, optimal performance, performance efficiency per power unit, and reduced communication costs, four novel global heuristic algorithms have been presented. These techniques first divide a linear program into several sub-blocks, then they generate a Sub-block Dependency Graph (SDG) that is then run on many cores [12]. To assign priority inside the SDG, the first technique, called the (Height Instruction) Count Based algorithm (HIB), combines instruction and height count computation. The HIB is a linear time technique that employs an effective search mechanism for scheduling each subblock on multicores. The HIB schedules each subblock as efficiently as possible. The second solution is the Dependent Sub-block Based Scheduler (DSB), which takes into account the reliance of subblocks and scheduling delay. SDG identifies all

alternative pathways from a given block to a leaf node and calculates the scheduling delay for each. Priority is given to the path with the greatest scheduling delay in the subblock. The third approach, known as Maximum Dependency Sub Block (MDS), prioritizes tasks with shorter execution times and higher dependency levels. Another method, Longest Latency Sub Block First (LLSF), focuses on selecting sub-blocks with the highest scheduling latency, sorting them in descending order of latency for scheduling [12]. Each node is associated with a sub-block and latency, which the scheduler uses to determine task allocation. The findings show that the HIB approach works well for less dense SDGs, meaning those with fewer child nodes. On the other hand, DSB and MDS aim to schedule dependent sub-blocks on the same core to reduce communication costs, making them more suitable for dense SDGs with limited cores. Among the two, DSB is more efficient during compile time compared to MDS. Although the LLSF approach provides better speedup than the other methods, it comes at the cost of higher power consumption and communication overhead. It is best suited for environments where maximizing performance is the top priority, regardless of resource efficiency [12].

When assigning tasks to partitions, several partitioning algorithms i.e., First-Fit, Best-Fit, and Worst-Fit are used, considering criticality and utilization [13]. The author employed the Dynamic Partitioning scheduling approach in [14]. Tasks are assigned to processors using the Dynamic Partitioning scheduling mechanism, and every task stays in the same core until it is completed. The slack time strategy, which partitions the task and assigns it to several cores rather than just one, is employed when a job cannot be given to any core. Every core uses the EDF algorithm in the Dynamic Partitioning scheduling technique. Priority is assigned according to a job's absolute deadline, with tasks having closer deadlines receiving higher priority. The Dynamic Partitioning-EDF strategy combines the advantages of global and partitioned scheduling techniques, aiming to reduce scheduling overhead while making better use of system resources. Compared to Global EDF, Dynamic Partitioning-EDF shows lower scheduling overhead and achieves higher CPU utilization.

Gu et al. [6] introduced the Mixed Criticality Partitioning with Virtual Deadlines (MPVD) approach for multicore platforms, building on a single-processor algorithm. The primary objective of MPVD is to enhance the system's schedulability by allocating tasks with different criticality levels to separate processors. It employs a hybrid strategy, using the Worst Fit (WF) method to assign high-criticality tasks and the First Fit (FF) method for low-criticality tasks. This combination ensures that deadlines for high-criticality tasks are rarely missed. However, MPVD's efficiency tends to decline as the number of processors increases. One noted challenge is that heavy low-criticality tasks may remain unassigned due to insufficient space on individual processors, even when the total capacity across all processors is adequate. To address this, resources are reserved for heavy low-criticality tasks

before high-criticality tasks are allocated, followed by an optimized virtual deadlines tuning algorithm to enhance MPVD's performance. Results indicate that MPVD outperforms existing algorithms in many scenarios.

Recently, cluster-based scheduling has been gaining traction in mixed criticality systems for implementation on multiprocessor platforms. The author categorizes the cores or processors into two groups for task allocation [8]. The first group consists of sub-clusters that feature smaller cores or processors, to which all tasks are assigned under low criticality conditions. The second group comprises clusters with larger cores or processors, designated for high critical tasks. The assignment of tasks to clusters and sub-clusters follows a worst fit heuristic strategy. For schedulability analysis, fixed priority response time assessment is employed, drawing from Audsley's [15] method. The results indicate that the cluster approach achieves a higher task scheduling efficiency than the Global and Partitioned scheduling strategies. Furthermore, the cluster-based method also shows a superior acceptance rate in comparison to both global and partitioned methods [8].

The authors in [16] proposed an algorithmic solution to enhance the allocation of heterogeneous servers to real-time tasks, aiming to maximize the number of tasks processed. By examining task and server characteristics within the mobile edge computing (MEC) architecture, the authors developed the suitability-based adaptive resource selection (SARS) algorithm, which assesses server suitability based on factors such as time constraints and server capabilities. Additionally, the authors introduced the proactive on-demand resource allocation (PORA) algorithm, which strategically reserves computational resources for critical real-time tasks [16]. The authors in [17] proposed a novel offloading strategy for MEC, Com-DDPG, utilizing multi-agent reinforcement learning to enhance offloading performance. Within the Internet of Vehicle (IoV) transmission radius, multiple agents collaborate to understand environmental changes, such as the number of mobile devices and task queues, and take appropriate actions for offloading to an edge server. The authors discussed models of task dependency, priority, and resource consumption from the perspective of server clusters and multiple task dependencies. The proposed method formulates the communication behavior among agents and executes the offloading strategy determined by reinforcement learning. To enhance communication among agents, a long short-term memory (LSTM) network is employed as an internal state predictor, and a bidirectional recurrent neural network (BRNN) is used to learn and enhance communication features [17].

Efficient task scheduling is crucial for optimizing performance in distributed computing systems. Scheduling tasks in heterogeneous distributed systems is a recognized NP-complete problem. While numerous algorithms have been developed for homogeneous systems, only a limited number address the challenges of heterogeneous environments. The Heterogeneous Earliest Finish Time (HEFT) algorithm

prioritizes tasks based on the highest upper rank and assigns them to the most suitable processing core. On the other hand, the Critical Path on a Processor (CPOP) algorithm leverages both downward and upward rank summations to determine task priority. A comparative analysis using graphs evaluates HEFT and CPOP against previous algorithms across four metrics. In terms of Schedule Length Ratio, HEFT slightly outperforms CPOP and the Dynamic Level Scheduling (DLS) algorithm. Additionally, HEFT shows a clear advantage over the Mapping Heuristic (MH) algorithm and is significantly more efficient than the Level Minimum Time (LMT) algorithm. For the Speedup metric, HEFT and CPOP demonstrate comparable performance, both surpassing MH, DLS, and LMT. When analyzing graph results, CPOP stands out for its superior performance in producing high-quality schedules more frequently. Finally, for Execution Time, HEFT is the fastest algorithm, while DLS is the slowest, taking the longest time to complete tasks [18].

A comparative analysis, visualized through graphs, evaluates HEFT and CPOP alongside earlier algorithms using four key metrics. For the Schedule Length Ratio metric, HEFT shows a slight advantage over CPOP and the Dynamic Level Scheduling (DLS) algorithm. Additionally, HEFT significantly outperforms the Mapping Heuristic (MH) algorithm and is notably more efficient than the Level Minimum Time (LMT) algorithm. When assessing the Speedup metric, HEFT and CPOP perform similarly and both surpass MH, DLS, and LMT. CPOP demonstrates exceptional performance by frequently producing high-quality schedules, as indicated by the graph analysis. Lastly, in terms of Execution Time, HEFT is the fastest algorithm, while DLS is the slowest, taking the longest time to complete its tasks [18].

Xie et al. [19] addressed two critical goals: minimizing makespan (the total length of the schedule) and reducing the deadline miss ratio (DMR). A shorter makespan enhances system performance, while a lower DMR ensures fewer high-criticality tasks miss their deadlines. To achieve these objectives, the authors introduced two novel algorithms: Fairness of Multiple Heterogeneous Earliest Finish Time (F-HMEFT) for optimizing makespan and Deadline Span of Multiple Heterogeneous Earliest Finish Time (D-MHEFT) for minimizing DMR. Results showed that these algorithms outperformed existing methods in terms of efficiency and reliability.

Despite extensive research in the field of distributed mixed-criticality systems, a significant gap remains in the effective handling of interfering tasks. Existing approaches often fall short in managing tasks that compete for shared resources, leading to increased waiting times and unresolved scheduling conflicts. Additionally, if interfering tasks are allocated to different cores, it may result in further increases in waiting time. This research aims to address these gaps by proposing a novel model designed to handle tasks that interfere with each other within a distributed platform. The proposed model focuses on minimizing the waiting time between these interfering tasks and effectively resolving scheduling conflicts, resulting in

increase in the schedulability of MC task sets. To achieve better schedulability, the approach allocates the maximum number of interfering tasks to the same core, thereby reducing waiting times and enhancing overall system performance and reliability.

III. SYSTEM MODEL

A model is essentially a simplified representation of an entity, focusing on its most critical features while excluding less relevant details. Created through a process of abstraction, models help manage complexity by highlighting only the essential aspects. The purpose of a model plays a key role in determining which features of the object are emphasized [20]. It provides a structural and design-based overview of the system's main components, offering a clear understanding of its core elements and how they interact [21]. Distributed mixed-criticality systems are designed to handle task sets with varying levels of criticality across multiple distributed nodes. These systems are widely used in mobile and embedded cloud computing environments, as well as in space-related applications. A key requirement for their effective operation is ensuring secure coordination of tasks across both temporal and spatial domains while meeting the specific timing constraints of each task [22]. Efficient task scheduling plays a vital role in maximizing system performance. This involves assigning tasks to the appropriate cores and determining the execution order for each resource to optimize utilization and timing. In our system model, multiple distributed nodes are interconnected via a Network-on-Chip (NoC) architecture, as depicted in Figure 1. This setup forms the foundation for coordinating and executing tasks efficiently across the distributed network.

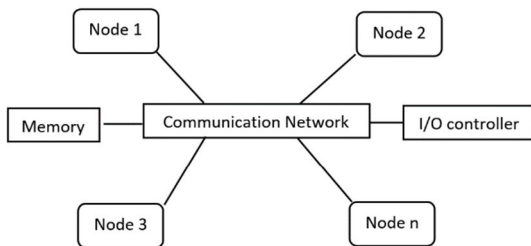


FIGURE 1. System model for distributed mixed criticality systems.

Each node in the system contains a set of cores, denoted as $m = \{C_1, C_2, C_3, \dots, C_m\}$, for processing a periodic task set $\Gamma = \{\tau_1, \tau_2, \tau_3, \dots, \tau_n\}$ of n MC tasks. Every core has access to a shared global memory, which is implemented as Dynamic Random Access Memory (DRAM). This DRAM is further organized into multiple memory blocks and banks, as explained by Georgia Giannopoulou et al. [23]. The node structure of the system model in a distributed mixed-criticality system is illustrated in Figure 2.

The task set Γ is made up of different superblocks, where a superblock is a group of real-time tasks that execute periodically on a processing element. These tasks share resources and have a defined computation time. In our system, we use

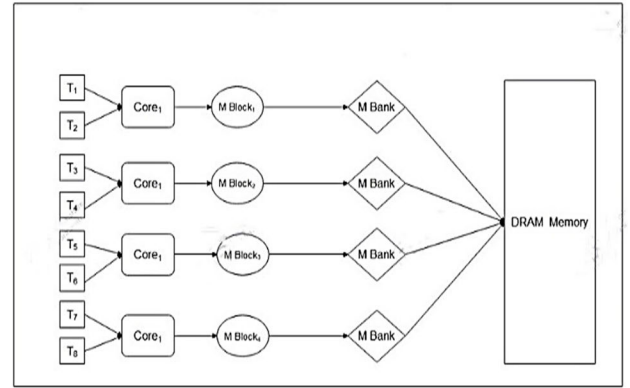


FIGURE 2. Node structure of system model in distributed mixed criticality systems.

a dedicated superblock model for memory access, where memory operations occur only during two specific phases: the acquisition phase and the replication phase. During the acquisition phase, data is retrieved from the shared global memory (DRAM). Once the data is acquired, the execution phase begins, in which computations are performed on the data. After the computations are completed, the replication phase starts, where the updated data is written back to the shared memory. Once this process is finished, the superblock is complete, and the memory is released for the next task in the waiting queue [3].

The system initially operates in low-criticality mode, where all tasks are executed based on their Worst Case Execution Time (WCET) for low mode. A transition to high-criticality mode occurs if a high-criticality task fails to complete within its allocated WCET in low mode. When this happens, low-criticality tasks are suspended, and the system begins operating using the WCET values designated for high-criticality mode [8]. Each core in the system has access to its dedicated memory block and is also connected to a memory bank and DRAM. Since tasks are distributed and share memory resources, they interfere with one another. When a task accesses the memory, other tasks must wait until the operation is completed and the memory is updated before they can proceed.

A. TASK MODEL

A task τ_i interacts with DRAM through memory blocks and memory banks. It follows a dedicated superblock model, which organizes the task into a sequence of computation and memory access phases, as depicted in Figure 3. The superblock execution model includes three distinct phases i.e., acquisition phase time (AP), execution phase time (C_i^x), and replication phase time (RP). During the AP phase, the task performs read or write operations in shared memory to acquire the necessary data. In the Execution Phase (EP), the task processes the fetched data. Finally, in the RP phase, the task updates the shared memory with the results of its computations. Once the task τ_i completes its execution, the shared memory becomes available for the next task. Each distributed periodic task $\tau_i = \{P_i, D_i, AP, C_i^{LO}, C_i^{HI}, RP, X_i, F_i\}$. Where:

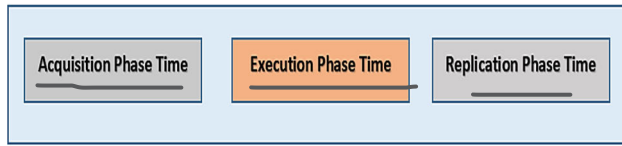


FIGURE 3. Dedicated super block for memory access.

TABLE 1. Mixed criticality task set.

τ_i	P_i	C_i^{LO}	C_i^{HI}	X_i	U_i^{LO}	U_i^{HI}	AC^{LO}	RP^{LO}	AC^{HI}	RP^{HI}	F_i	$UTot_i^{LO}$	$UTot_i^{HI}$
τ_1	5	2	0	LO	0.4	0	0.2	0.2	0.0	0.0	2	0.48	0
τ_2	4	1	2	HI	0.25	0.5	0.1	0.1	0.2	0.2	1	0.3	0.6
τ_3	20	3	0	LO	0.15	0	0.3	0.3	0.0	0.0	3	0.18	0
τ_4	8	1	3	HI	0.13	0.38	0.1	0.1	0.3	0.3	1	0.15	0.45
τ_5	8	1	0	LO	0.13	0	0.1	0.1	0.0	0.0	4	0.15	0
τ_6	40	4	0	LO	0.1	0	0.4	0.4	0.0	0.0	1	0.12	0
τ_7	20	4	5	HI	0.2	0.25	0.4	0.4	0.5	0.5	3	0.24	0.3
τ_8	4	1	0	LO	0.25	0	0.1	0.1	0.0	0.0	1	0.3	0
τ_9	5	1	2	HI	0.2	0.4	0.1	0.1	0.2	0.2	3	0.24	0.48
τ_{10}	40	6	0	LO	0.15	0	0.6	0.6	0.0	0.0	4	0.18	0
τ_{11}	8	1	0	LO	0.13	0	0.1	0.1	0.0	0.0	2	0.15	0
τ_{12}	10	2	0	LO	0.2	0	0.2	0.2	0.0	0.0	3	0.24	0
τ_{13}	10	3	4	HI	0.3	0.4	0.3	0.3	0.4	0.4	4	0.36	0.48
τ_{14}	4	1	2	HI	0.25	0.5	0.1	0.1	0.2	0.2	4	0.3	0.6
τ_{15}	10	1	3	HI	0.1	0.3	0.1	0.1	0.3	0.3	2	0.12	0.36
τ_{16}	5	1	2	HI	0.2	0.4	0.1	0.1	0.2	0.2	2	0.24	0.48

- P_i is the period of the task, specifying the minimum time interval between the release of consecutive jobs for τ_i .
- D_i is the deadline of the task, which is equal to its period.
- AP is the Acquisition Phase time, during which the task reads or writes data from the global memory.
- C_i^{LO} = The Worst Case Execution Time (WCET) of the task in low-criticality mode.
- C_i^{HI} = The WCET of the task in high-criticality mode.
- RP = The Replication Phase time, where data is written back to the global memory. After the acquisition and execution phases, the results are updated in this phase.
- X_i = The criticality level of the task, which can be either LO (low-criticality) or HI (high-criticality).
- F_i = The flag value, an integer that helps identify interfering tasks. Tasks with the same flag value are considered to interfere with each other. The flag value is an integer, and tasks with identical flag values are referred to as interfering tasks

IV. MOTIVATING EXAMPLE

In scheduling mixed-criticality distributed systems, common approaches include global, partitioned, and cluster scheduling. For the execution of the motivating example, we adopt the partitioned scheduling approach, specifically the Non-Interference Aware Partitioning-Fixed Priority

(NIAP-FP) method. Consider a distributed mixed-criticality system with sixteen tasks and four processing cores, where the tasks are represented as $T = \{\tau_1, \tau_2, \tau_3, \tau_4, \tau_5, \tau_6, \tau_7, \tau_8, \tau_9, \tau_{10}, \tau_{11}, \tau_{12}, \tau_{13}, \tau_{14}, \tau_{15}, \tau_{16}\}$, and the cores as $m = C_1, C_2, C_3, C_4$.

The acquisition and replication times are calculated as $AC = C_i^{LO} * 10\%$ and $RP = C_i^{LO} * 10\%$, respectively. The task's utilization in low-criticality mode is given by $U_i^{LO} = C_i^{LO}/P_i$, while in high-criticality mode it is $U_i^{HI} = C_i^{HI}/P_i$. For each core, the utilization at low-criticality mode is denoted by UC_k^{LO} , and at high-criticality mode by UC_k^{HI} . All these parameters are summarized in Table 1. A task set is considered schedulable in low-criticality mode if the Worst-Case Response Time ($WCRT_i^{LO}$) of each task τ_i is less than or equal to its deadline or period in low mode. Similarly, in high-criticality mode, the task set is deemed schedulable if the Worst-Case Response Time ($WCRT_i^{HI}$) of each high-criticality task τ_i is less than or equal to its deadline.

Example 1: Consider the system parameters provided in Table 1 for low-criticality mode. The process begins with a schedulability test for each task, ensuring that the total utilization of all tasks is less than or equal to the total number of cores. If the task set passes the schedulability test, the utilization for each task is calculated. Tasks are then assigned priorities based on the Decreasing Criticality and Decreasing Utilization (DCDU) approach [24]. According to this method, tasks are ordered by first considering their criticality levels, followed by their utilization values in descending order as $\tau_9, \tau_{16}, \tau_4, \tau_{13}, \tau_{15}, \tau_3, \tau_7, \tau_6, \tau_2, \tau_8, \tau_{10}, \tau_{14}, \tau_1, \tau_{12}, \tau_5, \tau_{11}$. After assigning priorities, high-criticality tasks are allocated to cores using the Worst Fit (WF) policy [24]. According to this policy, tasks are assigned to the core with the most available free space. Initially, since all cores have zero utilization, tasks $\tau_9, \tau_{16}, \tau_4$, and τ_{13} are allocated to cores C_1, C_2, C_3 and C_4 , respectively. After this first round of allocation, the utilizations of the cores in low mode are updated as follows: $UC_1^{HI} = 0.5$, $UC_2^{HI} = 0.5$, $UC_3^{HI} = 0.4$, and $UC_4^{HI} = 0.4$. The next task in the priority queue is allocated to the core with the lowest utilization or, in other words, the core with the most-free space. Among the cores, $UC_4^{HI} = 0.36$, have the lowest utilizations. Thus, tasks τ_{15} is assigned to C_4 . The updated utilizations is $UC_4^{HI} = 0.36 + 0.36 = 0.72$. The next task, τ_3 is allocated to C_3 , as it now has the lowest utilization among the remaining cores. The utilization of C_3 is increased by 0.3, resulting in $UC_3^{HI} = 0.45 + 0.3 = 0.75$. The second last high-criticality task τ_7 is assigned to C_1 , whose utilization is updated to $UC_1^{HI} = 0.78$. The last high critical task τ_6 is assigned to C_2 , and the updated value of UC_2^{HI} is 0.72. The final allocation of high-criticality tasks to cores is illustrated in Figure 4.

After the allocation of high critical tasks in high criticality mode, now it's the turn of low critical tasks to be allocated in low criticality mode. In low criticality mode the utilization of four cores are $UC_1^{LO} = 0.48$, $UC_2^{LO} = 0.36$, $UC_3^{LO} = 0.33$, and $UC_4^{LO} = 0.48$. Among the four cores now it's the turn of C_3 , so the next task τ_2 in low criticality mode is added

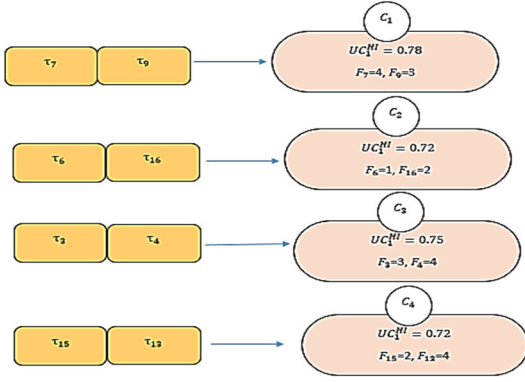


FIGURE 4. Task allocation to cores in high mode using DCDU-WF approach.

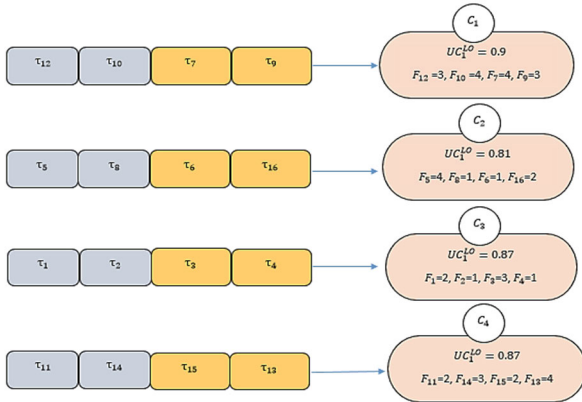


FIGURE 5. Task allocation to cores in Low mode using DCDU-WF approach.

to C_3 and its utilization becomes $UC_3^{LO} = 0.33+0.3=0.63$. The upcoming task τ_8 is allocated to C_2 and its utilization $UC_2^{LO} = 0.36+0.3=0.66$. Task τ_{10} is added to C_1 and its utilization becomes 0.66. The task τ_{14} , is assigned to C_4 and $UC_4^{LO} = 0.48+0.24=0.72$. In the priority queue, now it's the turn of τ_1 which is given to C_3 and making its utilization $UC_3^{LO} = 0.63+0.24=0.87$. In the same way task τ_{12} is assigned to C_1 and $UC_1^{LO} = 0.66+0.24=0.9$. The task τ_5 is allocated to the core C_2 , making the $UC_2^{LO} = 0.81$. The last task in the priority queue is τ_{11} which is moved to C_4 , whose utilization UC_4^{LO} is increased to 0.87. The task allocation across cores in low-criticality mode is depicted in Figure 5.

The task set is scheduled on each core using a fixed-priority partitioned approach. Task priorities are determined using Audsley's optimal priority assignment method, which ensures that the response time of every task on its respective core remains within its deadline. Audsley's algorithm assigns priorities in a recursive manner, starting with the lowest priority and progressing to the highest. During this process, if a task is assigned a lower priority, it is removed from the pool of tasks awaiting priority assignment. However, if no suitable task can be found for the next priority level during the recursion, it indicates that the task set cannot be scheduled using the fixed-priority response time analysis

method. This approach is systematic and ensures that tasks are prioritized in a way that maximizes schedulability within the given constraints.

Schedulability analysis is performed using the Worst-Case Response Time $WCRT_i^X$ for each task. The $WCRT_i^{LO}$ and $WCRT_i^{HI}$ represent the worst-case response times for a task τ_i in low-criticality and high-criticality modes, respectively. These values are calculated using Equations 1 and 2 and are summarized in Table 2.

$$WCRT_i^{LO} = \sum_{j \in hp}^n WCRT_j^{LO} + C_{ij}^{LO} + \text{Mint} * \text{AP} * \text{Tac} + \text{Mint} * \text{RP} * \text{Tac} \quad (1)$$

$$WCRT_i^{HI} = \sum_{j \in hp}^{nHi} WCRT_j^{HI} + C_{ij}^{HI} + \text{Mint} * \text{AP} * \text{Tac} + \text{Mint} * \text{RP} * \text{Tac} \quad (2)$$

In this context, Mint represents the number of cores where at least one interfering task is actively running, while Tac denotes the latency time, which is set to 0.6 ms. The parameter $\sum_{j \in hp}^n$ accounts for the sum of all higher-priority tasks than τ_i , and $\sum_{j \in hp}^{nHi}$ refers to the sum of all higher-priority tasks with high criticality. The Worst-Case Response Time (WCRT) for both low-criticality and high-criticality modes is determined using the Audsley priority assignment approach, as shown in Table 2. The table reveals that tasks τ_3 and τ_4 on core C_3 , and task τ_{13} on C_4 fail to meet their respective deadlines in low criticality mode, indicating that the task set is not schedulable.

After the system switches to high-criticality mode, the WCRT approach based on Audsley priority assignment approach successfully schedules the high critical tasks in high criticality mode as shown in Table 2, with tasks allocated as follows: $C_1 = \{\tau_9, \tau_7\}$, $C_2 = \{\tau_{16}, \tau_6\}$, $C_3 = \{\tau_4, \tau_3\}$, and $C_4 = \{\tau_{15}, \tau_{13}\}$. All high-criticality tasks meet their deadlines in this mode. The proposed partitioned approach demonstrates its capability to schedule tasks effectively in both low-criticality and high-criticality modes, as detailed in the subsequent sections.

V. PROPOSED TASK ALLOCATION FOR DISTRIBUTED MIXED CRITICALITY SYSTEMS

In traditional mixed-criticality systems, task priority is typically assigned based on decreasing criticality and decreasing utilization for the allocation of tasks to cores. However, such systems are often either non-distributed or fail to account for task interference. To tackle the issue of waiting time of interfering tasks, this research work proposed a novel method named Interference-Aware Decreasing Criticality and Decreasing Utilization with Worst-Fit (IADCDCU-WF) which allocate mixed-criticality tasks to cores in distributed systems.

A. PRIORITY ASSIGNMENT AND ALLOCATION OF TASKS TO GROUP

In mixed-criticality systems, tasks are prioritized based on the principle of Decreasing Criticality and Decreasing Utilization

TABLE 2. Response time of tasks.

C_i	T_i	$P_i = D_i$	$WCRT_i^{LO}$	$WCRT_i^{HI}$
C_1	τ_9	5	1.2	2.4
	τ_{12}	10	3.6	
	τ_7	20	8.4	8.4
	τ_{10}	40	36	
C_2	τ_8	4	1.2	
	τ_{16}	5	2.4	2.4
	τ_5	8	3.6	
	τ_6	40	32.4	12.0
C_3	τ_2	4	2.4	
	τ_1	5	3.6	
	τ_4	8	9.6	3.6
	τ_3	20	23.4	9.6
C_4	τ_{14}	5	3.6	
	τ_{11}	8	4.8	
	τ_{15}	10	7.2	3.6
	τ_{13}	10	10.6	7.2

(DCDU). This approach prioritizes tasks with higher criticality over those with lower criticality. Among high-criticality tasks, those with greater utilization may be given precedence over other high-criticality tasks. Similarly, low-criticality tasks are arranged in order of decreasing utilization. The utilization of a task τ_i , represented as U_i , is calculated by dividing its worst-case execution time by its period. This relationship can be expressed mathematically as follows:

$$U_i^{LO} = C_i^{LO} / P_i \quad (3)$$

$$U_i^{HI} = C_i^{HI} / P_i \quad (4)$$

Once tasks are assigned priority then using Worst Fit (WF) heuristic policy for the allocation of tasks to different Groups. The heuristic approach allocates a task to the group having the largest unused space among the other groups. The UG^{HM} is used to denote the high criticality task utilization of a group in high mode which is equal to the sum of high mode utilizations of high criticality tasks in the group. The WF heuristic distributes the workload of highly critical tasks more uniformly across the groups. As a result, the groups for all Cores are effectively utilized for scheduling high criticality tasks in high mode. The UG_k^{LM} represents the low criticality task utilization of a group in low criticality mode. The utilization of a Group G_k in low mode (UG_k^{LM}) is also considered for high criticality tasks. After assigning the high criticality tasks to a Group, the utilization UG_k^{LM} is equal to the sum of low mode utilizations of high criticality tasks in the Group. Now using the DCDU priority order, each low criticality (LC) task is assigned to a Group G_k with the smallest utilization in low mode.

Let us take the task set available in Table 2. The priorities are assigned to tasks using DCDU. In the motivational example, we have eight high critical tasks, based on DCDU in high mode U_i^{HI} , they are given priority and are ordered as: τ_2 , τ_{14} , τ_9 , τ_{16} , τ_4 , τ_{13} , τ_{15} , τ_7 . Low critical tasks

are ordered through DCDU, using U_i^{LO} , in low criticality mode as, τ_1 , τ_8 , τ_{12} , τ_{10} , τ_3 , τ_5 , τ_{11} , τ_6 . The total utilization of task set in high criticality mode and low criticality mode is 3.125 which is less than the total number of cores, which is 4. Initially, the utilization of all four groups G_1 , G_2 , G_3 and G_4 for high criticality tasks in high criticality mode is zero, therefore high critical tasks are allocated to groups G_1 , G_2 , G_3 and G_4 respectively using WF policy. The high mode utilization of G_1 , G_2 , G_3 and G_4 is UG_1^{HM} , UG_2^{HM} , UG_3^{HM} , and UG_4^{HM} respectively and its values becomes 0.8, 0.75, 0.775 and 0.8. while low mode utilization of these four groups UG_1^{LM} , UG_2^{LM} , UG_3^{LM} and UG_4^{LM} becomes 0.35, 0.45, 0.325 and 0.5 respectively. For low criticality task allocation in low criticality mode G_3 has the smallest task utilization which is 0.325, so low criticality task τ_1 is allocated to G_3 . After allocating τ_1 to G_3 , the utilization of G_3 , is UG_3^{LM} which becomes 0.725. Now G_1 has the smallest low mode utilization i.e. 0.35, so the next task τ_8 is allocated to G_1 and its utilization value increases to 0.60. The next task in low criticality mode is τ_{12} which is assigned to G_1 whose utilization is 0.45 and its utilization becomes 0.65, while τ_{10} is allocated to G_4 whose utilization becomes 0.675. after allocating the first four low critical tasks to each group, their utilization is $UG_1^{LM} = 0.6$, $UG_2^{LM} = 0.65$, $UG_3^{LM} = 0.725$ and $UG_4^{LM} = 0.675$. Among these four groups G_1 has the minimum utilization so the next task in the priority queue is τ_3 , which is assigned to the group G_1 whose utilization UG_1^{LM} becomes 0.75. After assignment of τ_3 to G_1 its utilization becomes 0.75. the next task τ_5 is allocated to G_2 whose utilization UG_2^{LM} becomes 0.775. The task τ_{11} is allocated to the group G_4 having minimum utilization among the other groups, the utilization value of G_4 is raised to 0.775, after task allocation. The last task in the priority queue is τ_6 , which is allocated to the group G_3 and the utilization of G_3 in low criticality mode is UG_3^{LM} which becomes 0.825. All the low and high critical tasks are allocated to four groups as shown in Figure 6. The algorithm for allocation of task to groups is given in algorithm 1.

B. FLAG COUNTING MODULE

The algorithm 2 is used to count the occurrences of a specific flag f_i within groups of tasks, denoted as G_k . It begins by initializing a counter $G_k.f_i_count$ to zero for each group. Then, for each group G_k , it iterates over all the tasks τ_i contained within that group. During this iteration, the algorithm checks if the flag value associated with each task $\tau_i.f_i$ matches the specific flag f_i being counted. If a match is found, the counter for that group is incremented. After processing all the tasks within the group, the algorithm returns the final count of how many times the flag appeared within that group. The result is a tally of flag occurrences for each group, useful in scenarios where tasks are categorized by groups and specific attributes or flags within those tasks need to be tracked.

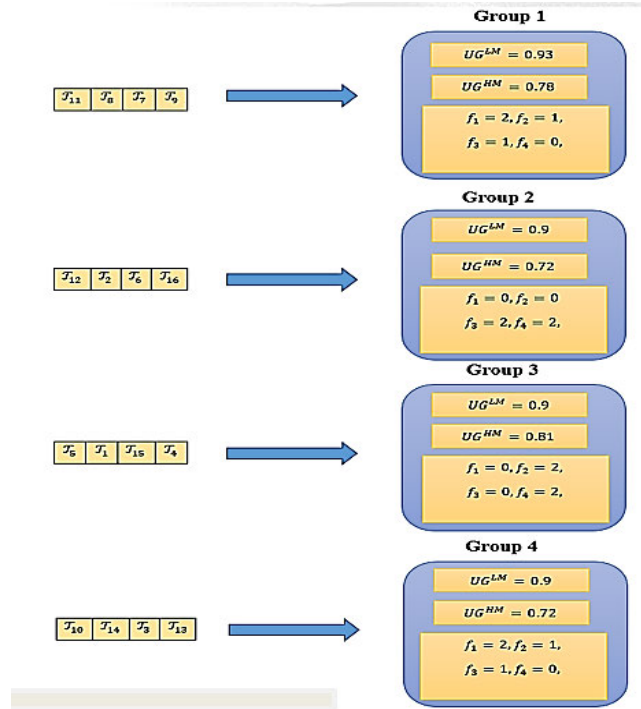
Let us consider the task set in table 1, the flag value of a task τ_i is represented by f_i , where $i = \{1, 2, 3, 4\}$. Tasks having the same flag values are interfering tasks. In group G_i the flag

Algorithm 1 Allocation of Tasks to Groups Through DCDU-WF

```

1.  $MinUG_k^{LM} = \infty$ 
2.  $MinUG_k^{HM} = \infty$ 
3. Sort the tasks in DCDU order.
/* allocating high critical tasks to group  $G_k$ , having smallest
used space  $UG_k^{HM}$  value */
4. For each high critical task  $\tau_i \in T$  do. /*  $T$  = task set. */
5.   For  $k=1$  to  $n_G$  do.
6.   If  $UG_k^{HM} < MinUG_k^{HM}$  then
7.      $MinUG_k^{HM} = UG_k^{HM}$ 
8.      $K = k$ 
9.    $G_k = G_k + \tau_i$ 
10.   $UG_k^{HM} = UG_k^{HM} + U_i^{HM}$  /* Update  $UG_k^{HM}$  value for
    group  $G_k$  */
/* allocating low critical tasks to group  $G_k$ , having smallest
used space  $UG_k^{LM}$  value */
11. For each low critical task  $\tau_i \in T$  do. /*  $T$  = task set. */
12.   For  $k=1$  to  $n_G$  do.
13.   If  $UG_k^{LM} < MinUG_k^{LM}$  then
14.      $MinUG_k^{LM} = UG_k^{LM}$ 
15.      $K = k$ 
16.    $G_k = G_k + \tau_i$ 
17.   $UG_k^{LM} = UG_k^{LM} + U_i^{LM}$  /* Update  $UG_k^{LM}$  value for
    group  $G_k$  */

```

**FIGURE 6.** Allocation of tasks to groups through DCDU-WF.

value f_i of a task τ_i is represented by $G_k.f_i_count$. In group G_1 the flag values for f_1, f_2, f_3 and f_4 are $G_1.f_1_count = 2$, $G_1.f_2_count = 1$, $G_1.f_3_count = 1$ and $G_1.f_4_count = 0$.

Algorithm 2 Count Number of Flag in Group

```

1.  $f_i$  = flag value
2.  $G_k.f_i\_count$  = number of flag in  $G_k$ 
3. For each  $G_k$ 
4.    $G_k.f_i\_count = \Phi$ 
5. For each task  $\tau_i$  in  $G_k$ 
6.   If  $\tau_i.f_i = f_i$ 
7.      $G_k.f_i\_count++$ 
8. Return  $G_k.f_i\_count$ 

```

For the group G_2 , the flag values for f_1, f_2, f_3 and f_4 are $G_2.f_1_count = 0$, $G_2.f_2_count = 0$, $G_2.f_3_count = 2$ and $G_2.f_4_count = 2$. For the third group G_3 the flag values for f_1, f_2, f_3 and f_4 are $G_3.f_1_count = 2$, $G_3.f_2_count = 1$, $G_3.f_3_count = 1$ and $G_3.f_4_count = 0$. The last group G_4 has flag values as $G_4.f_1_count = 0$, $G_4.f_2_count = 2$, $G_4.f_3_count = 0$ and $G_4.f_4_count = 2$, for the flag count f_1, f_2, f_3 and f_4 respectively.

C. SWAPPING OF TASKS FOR HIGH CRITICALITY MODE

The algorithm 3 focuses on swapping high criticality tasks between groups based on their flag values, aiming to balance the distribution of tasks with a similar flag across different groups. The process starts by maintaining a list of groups $G_1, G_2, G_3, \dots, G_m$ and sorting the tasks within each group based on the number of occurrences of a specific flag F_i_count . The algorithm then identifies the groups with the maximum and minimum counts of high criticality tasks associated with the flag. It does this by iterating through all the groups and comparing each group's flag count with the current maximum and minimum values. If a group's flag count exceeds the current maximum, it becomes the new maximum group, Max.G, and similarly, if a group's flag count is lower than the current minimum, it becomes the new minimum group, Min.G.

Once the maximum and minimum groups are identified, the algorithm proceeds to examine the high criticality tasks within the group that has the maximum flag count. It then updates the maximum flag count for that group. This step implies that the next phase would involve swapping tasks between the groups to balance the flag occurrences. The overarching goal is to ensure that tasks with similar flags are more evenly distributed across the groups, particularly focusing on those tasks deemed to be of high criticality.

The algorithm 4 is concerned with swapping high criticality tasks between groups to balance the distribution of flagged tasks. It begins by iterating over the high criticality tasks within the group identified as having the minimum flag count Min.G. For each task in Min.G, it updates the utility values of the maximum Max.G and minimum groups by removing the utility contribution of the task being swapped and adding the utility of the task from the other group. This updated utility is denoted by U_M^{HI} for both groups.

Algorithm 3 Swapping of Tasks Having Similar Flag in Groups for High Mode

```

*/ swapping of high criticality tasks */
1. List = {G1 G2 G3 ..... Gm}
2. M.List = List
3. Sorted_Gk = sort task in group by Fi_count
4. Max. Fi_count(HI) = 0
5. Min. Fi_count(HI) = 0
6. For each group G1 to Gm ∈ M.List
7. {
8.     IF Max. Fi_count(HI) < Fi_count(HI) then
9.     {
10.        Max. Fi_count(HI) = Fi_count(HI)
11.        Max.G = Gi
12.    }
13.    IF Min. Fi_count(HI) > Fi_count(HI) then
14.    {
15.        Min. Fi_count(HI) = Fi_count(HI)
16.        Min.G = Gi
17.    }
18. Max. Fi_count = 0
19. For each high task ∈ Max.G
20. {
21.     If Max. Fi_count < Max.G. Fi_count
22.     Max. Fi_count = Max.G. Fi_count
23. }
24. Calling algorithm 4

```

The algorithm then checks whether the utility difference between the two groups Max.G and Min.G, after the potential swap is below a certain threshold V, and whether the utility values of both groups are less than 1. If these conditions are satisfied, the tasks τ_j from Max.G and τ_i from Min.G are swapped. This swap is performed to balance the utility between the groups.

If the utility of the maximum group after the swap is less than 1 and the difference between the utilities of Max.G and Min.G remains within the threshold 'V', the maximum group is removed from the list of groups M.List. Similarly, if the utility of the minimum group falls below 1 and the utility difference meets the threshold, the minimum group is also removed from the list. The algorithm continues this process, ensuring that high criticality tasks are redistributed in a way that balances the load of flagged tasks across the groups.

Let us consider the above algorithm, initially Max. $F_{i_count}(HI)$ and Min. $F_{i_count}(HI)$ for all groups are zero. First Max. $F_{i_count}(HI)$ and Min. $F_{i_count}(HI)$ for each group is calculated. The group G_1 has Max. $F_{i_count}(HI) = 1$ and Min. $F_{i_count}(HI) = 2$. For G_2 Max. $F_{i_count}(HI) = 4$ and Min. $F_{i_count}(HI) = 3$, for G_3 Max. $F_{i_count}(HI) = 3$ and Min. $F_{i_count}(HI) = 2$. for G_4 Max. $F_{i_count}(HI) = 2$ and Min. $F_{i_count}(HI) = 4$. The group G_1 iteratively checks all the remaining groups for Min. $F_{i_count}(HI) = 1$, in group G_3 the task τ_4 has Min. $F_{i_count}(HI) = 1$, so it will

Algorithm 4 Checking the Condition for Swapping of Tasks in Groups for High Mode

```

1. For each high task ∈ Min.G
2. {
3.    $U_M^{HI}.Max.G = U^{HI}.Max.G_j U^{HI}.\tau_j^{Max.G} + U^{HI}.\tau_i^{Min.G}$ 
4.    $U_M^{HI}.Min.G = U^{HI}.Min.G_j U^{HI}.\tau_i^{Min.G} + U^{HI}.\tau_j^{Max.G}$ 
5.   If  $((U_M^{HI}.Max.G \&\& U_M^{HI}.Min.G) < 1) \&\& ((U_M^{HI}.Max.G - U_M^{HI}.Min.G) \leq V)$  then
6.     Swap  $(\tau_j^{Max.G}, \tau_i^{Min.G})$ 
7.   If  $((U_M^{HI}.Max.G < 1) \&\& ((U_M^{HI}.Max.G - U_M^{HI}.Min.G) \leq V))$  then
8.     M.List = M.List - Max.G
9.   If  $((U_M^{HI}.Min.G < 1) \&\& ((U_M^{HI}.Max.G - U_M^{HI}.Min.G) \leq V))$  then
10.    M.List = M.List - Min.G
11. }
12. }

```

be swapped with the task τ_{15} of the group G_1 while satisfying the two conditions i.e. $U_M^{HI}.Max.G_1 \&\& U_M^{HI}.Min.G_3 < 1$ and $U_M^{HI}.Max.G_1 - U_M^{HI}.Min.G_3 \leq V$. The condition $U_M^{HI}.Max.G \&\& U_M^{HI}.Min.G < 1$ means that the sum of the utilization of τ_1 and τ_4 while sum of the utilization of τ_9 and τ_{15} should be less than 1. The second condition $U_M^{HI}.Max.G - U_M^{HI}.Min.G \leq V$ shows that the difference between total utilization of group G_1 and G_3 should be less than or equal to the threshold value. The swapping satisfies the above two mentioned conditions so τ_{15} of G_1 and τ_4 of G_3 are swapped. The group G_2 searches for Min. $F_{i_count}(HI) = 4$ and τ_{13} of G_4 has Min. $F_{i_count}(HI) = 4$, so τ_7 of G_2 and τ_{13} of G_4 are swapped because they satisfy both the conditions that is $U_M^{HI}.Max.G_2 = 0.8 \&\& U_M^{HI}.Min.G_4 = 0.6 < 1$ and $U_M^{HI}.Max.G_2 - U_M^{HI}.Min.G_4 \leq V$. The group G_3 has Max. $F_{i_count}(HI) = 3$, so G_3 searches for Min. $F_{i_count}(HI) = 3$, the task τ_7 of G_4 has Min. $F_{i_count}(HI) = 3$, hence τ_{15} of G_3 will be swapped with τ_7 while meeting the two conditions, swapping is done because they meet the two conditions. Similarly, the last group G_4 tasks are τ_{16} and τ_{15} whose flag values are the same i.e. 2 and we don't need to swap them.

D. SWAPPING OF TASKS FOR BALANCING WORKLOAD UTILIZATION

In high criticality mode, when tasks are exchanged to equalize flag values of utilization, the workload utilization in low mode could surpass 1. In addressing this concern, Algorithm 5 aims to equilibrate the workload utilization across groups, thereby ensuring that the total utilization in low mode remains at or below 1 for each respective group. The algorithm functions on a list of task groups, represented as $\{G_1, G_2, G_3, \dots, G_m\}$, where the average utilization in low mode is computed across all groups. If a group's utilization is within 5% of the average, that group is omitted from further modifications. The algorithm then determines the group with maximum low mode utilization $Max.UG^{LO}$ and

the one with the minimum low mode utilization $Min.UG^{LO}$ from the remaining groups. It continues by selecting the task with the greatest utilization from the group exhibiting the highest workload utilization and executes task swaps to sustain balanced utilization throughout all groups.

Algorithm 5 Swapping of Tasks to Balance the Utilization of Tasks in Groups in Low Mode

```

1. List = {G1, G2, G3, . . . . . Gm}
2. M.List = List
3. Avg.  $UG_{i \rightarrow m}^{LO} = (\sum_{i=1}^m UG_i^{LO})/m$ 
4. For each group G1 to Gm
5.   If Avg.  $UG_{i \rightarrow m}^{LO} - 0.05 \leq UG_i^{LO} \leq$  Avg.  $UG_{i \rightarrow m}^{LO} + 0.05$  then
6.     M.List = List - Gi
7. End for
8. Max.  $UG^{LO} = 0$ 
9. Min.  $UG^{LO} = 0$ 
10. For each group G1 to Gm ∈ M.List
11. {
12.   If Max.  $UG^{LO} < UG_i^{LO}$  then
13.   {
14.     Max.  $UG^{LO} = UG_i^{LO}$ 
15.     Max.G = Gi
16.   }
17.   If Min.  $UG^{LO} > UG_i^{LO}$  then
18.   {
19.     Min.  $UG^{LO} = UG_i^{LO}$ 
20.     Min.G = Gi
21.   }
22. Max.  $UG_i^{LO} = 0$ 
23. For each low task ∈ Max.G
24. {
25.   If Max.  $UG_i^{LO} < Max.G. U_i^{LO}$ 
26.   Max.  $U_i^{LO} = Max.G. U_i^{LO}$ 
27. }
```

E. SWAPPING OF TASKS FOR LOW CRITICALITY MODE

The algorithm 6 is intended to swap low criticality tasks between groups based on their flag values, in order to balance the distribution of these tasks. It starts by iterating through each group G₁, G₂, G₃, G_m in a list M.List. For each group, the algorithm compares the flag counts for low criticality tasks $F_{i_count}(LO)$. If a group has a flag count higher than the current maximum, it updates the maximum flag count Max. $F_{i_count}(LO)$ and sets that group as the maximum group Max.G. Conversely, if a group has a lower flag count than the current minimum, it updates the minimum flag count Min. $F_{i_count}(LO)$ and designates that group as the minimum group Min.G.

After identifying the groups with the highest and lowest counts of low criticality flagged tasks, the algorithm proceeds to reset Max. F_{i_count} to zero and iterates through the tasks in

the maximum group. It then updates Max. F_{i_count} with the highest flag count for low criticality tasks within that group.

This process is aimed at redistributing low criticality tasks between groups by swapping tasks between the maximum and minimum groups in order to achieve a more balanced distribution of flagged tasks. Though the swapping mechanism itself is not detailed in this excerpt, the purpose is to ensure that no group is overloaded or under loaded with low criticality tasks.

Algorithm 6 Swapping of Tasks Having Similar Flag in Groups for Low Mode

```

*/ swapping of low criticality task */
1. For each group G1 to Gm ∈ M.List
2. {
3.   IF Max.  $F_{i\_count}(LO) < F_{i\_count}(LO)$  then
4.   {
5.     Max.  $F_{i\_count}(LO) = F_{i\_count}(LO)$ 
6.     Max.G = Gi
7.   }
8.   IF Min.  $F_{i\_count}(LO) > F_{i\_count}(LO)$  then
9.   {
10.    Min.  $F_{i\_count}(LO) = F_{i\_count}(LO)$ 
11.    Min.G = Gi
12.  }
13. Max.  $F_{i\_count} = 0$ 
14. For each high task ∈ Max.G
15. {
16.   If Max.  $F_{i\_count} < Max.G. F_{i\_count}$ 
17.   Max.  $F_{i\_count} = Max.G. F_{i\_count}$ 
18. }
19. Calling algorithm 7
```

The algorithm 7 focuses on swapping low criticality tasks between two groups, Max.G and Min.G, based on their utility values. The goal is to balance the utility of these tasks between the groups, ensuring no group is over- or under-loaded with low criticality tasks.

The process begins by iterating over the high criticality tasks in the group with the lowest utility Min.G. For each task in Min.G, the algorithm recalculates the utility of both the maximum group Max.G and the minimum group Min.G after a potential swap. The utility of the maximum group U_M^{LO} .Max.G is updated by subtracting the utility of the task being swapped from Max.G and adding the utility of the task from Min.G. Similarly, the utility of the minimum group U_M^{LO} .Min.G is recalculated by adjusting for the tasks involved in the swap.

Once the new utilities are calculated, the algorithm checks two conditions: first, that the utilities of both Max.G and Min.G are less than 1, and second, that the difference between the utilities of the two groups is less than or equal to a threshold V. If these conditions are met, the tasks τ_j from Max.G and τ_i from Min.G are swapped to balance the groups.

Further checks are performed after the swap. If the utility of the maximum group $U_M^{LO}.MaxG$ falls below 1 and the difference between Max.G and Min.G remains within the threshold V, the maximum group is removed from the list of groups M.List. The same process is applied to the minimum group if its utility falls below 1. In essence, this algorithm ensures a fair distribution of low criticality tasks by continuously swapping tasks between groups until a balance is achieved, preventing any one group from being overloaded or underutilized.

Algorithm 7 Checking the Condition for Swapping of Tasks in Groups for Low Mode

1. For each low task $\in$ Min.G
2. {
3. $U_M^{LO}.Max.G = U_M^{LO}.Max.G - U^{LO}.\tau_j^{Max.G}$
 $+ U^{LO}.\tau_i^{Min.G}$
4. $U_M^{LO}.Min.G = U_M^{LO}.Min.G_j - U^{LO}.\tau_i^{Min.G}$
 $+ U^{LO}.\tau_j^{Max.G}$
5. If $((U_M^{LO}.Max.G \& \& U_M^{LO}.Min.G) < 1) \& \& ((U_M^{LO}.Max.G - U_M^{LO}.Min.G) \leq V)$ then
6. Swap $(\tau_j^{Max.G}, \tau_i^{Min.G})$
7. If $((U_M^{LO}.Max.G < 1) \& \& ((U_M^{LO}.Max.G - U_M^{LO}.Min.G) \leq V))$ then
8. M.List = M.List – Max.G
9. If $((U_M^{LO}.Min.G < 1) \& \& ((U_M^{LO}.Max.G - U_M^{LO}.Min.G) \leq V))$ then
10. M.List = M.List – Min.G
11. }
12. }

Algorithm 6 and algorithm 7 consider the allocation of low criticality tasks in low criticality mode. Initially Max. $F_{i_count}(LO)$ and Min. $F_{i_count}(LO)$ for all groups are zero in low criticality mode. First Max. $F_{i_count}(LO)$ and Min. $F_{i_count}(LO)$ for each group is calculated. The group G1 has Max. $F_{i_count}(LO) = 1$ and Min. $F_{i_count}(LO) = 3$. For G2 Max. $F_{i_count}(HI) = 4$ and Min. $F_{i_count}(HI) = 1$, for G3 Max. $F_{i_count}(HI) = 3$ and Min. $F_{i_count}(HI) = 2$. For G4 Max. $F_{i_count}(HI) = 2$ and Min. $F_{i_count}(HI) = 4$. The group G1 iteratively checks all the remaining groups for Min. $F_{i_count}(HI) = 1$, in group G2 the task τ_6 has Min. $F_{i_count}(HI) = 1$, so it will be swapped with the task τ_3 of the group G1 while satisfying the two conditions i.e. $U_M^{HI}.Max.G1 \& \& U_M^{HI}.Min.G3 < 1$ and $U_M^{HI}.Max.G1 - U_M^{HI}.Min.G3 \leq V$. The group G2 searches for Min. $F_{i_count}(HI) = 4$ and τ_{10} of G3 has Min. $F_{i_count}(HI) = 4$, so τ_3 of G2 and τ_{10} of G3 are swapped because they satisfy both conditions. After swapping of τ_3 and τ_{10} , the group G3 has automatically gained the Max. $F_{i_count}(LO) = 3$ and Min. $F_{i_count}(LO) = 0$. The same interfering tasks are already assigned into G4 in the balance utilization phase. Interfering tasks assigned to the same groups are shown in figure 7.

VI. SCHEDULING OF TASKS IN DISTRIBUTED MIXED CRITICALITY SYSTEMS

Scheduling in mixed-criticality systems involves allocating resources and determining the execution order of tasks to ensure that the timing constraints of tasks with varying criticality levels are met. A key challenge in scheduling distributed mixed-criticality systems is that existing methods often fail to effectively address the interference between tasks of different criticality levels sharing the same processing resources. While fairness policies are often employed to minimize the overall completion time in distributed systems, they may not always guarantee that the majority of tasks meet their deadlines. For scheduling, we have proposed a novel Interference aware Partitioning-Fixed Priority (IAP-FP) algorithm based on worst case response time approach using Audsley priority assignment.

A. SCHEDULABILITY TEST

Before scheduling a task set using the proposed method, it is important to first verify whether the tasks can actually be scheduled. A task τ_i is considered schedulable in low mode if its worst-case response time in low mode $WCRT_i^{LO}$ is less than or equal to its deadline in low mode D_i . Similarly, a high-priority task τ_i is deemed schedulable in high mode if its worst-case response time in high mode $WCRT_i^{HI}$ does not exceed its deadline in high mode. Likewise, a high criticality task τ_i is schedulable in mode change if its WCRT during mode change $WCRT_i^{MC}$ does not exceed its deadline D_i . The schedulability test for low criticality mode, high criticality mode and mode change is presented in equation 5, equation 6 and equation 7 respectively.

$$\forall \tau_{iLO}; WCRT_i^{LO} \leq D_i \quad (5)$$

$$\forall \tau_{iHI}; WCRT_i^{HI} \leq D_i \quad (6)$$

$$\forall \tau_{iHI}; WCRT_i^{MC} \leq D_i \quad (7)$$

If a task set meets the conditions outlined in the equations above, it is considered schedulable. However, if these conditions are not met, the task set is deemed non-schedulable.

B. SCHEDULING OF TASKS IN LOW CRITICALITY MODE

The tasks are scheduled in low criticality mode using Interference Aware Partitioned Fixed Priority Scheduling (IAP-FP). In the following scheduling figures, each task is represented by a different color. The vertical arrows with downward pointing heads show period of tasks and the arrows with upward pointing heads show the deadline of tasks. Vertical arrows with both upward and downward heads show that these tasks have the same period and deadline. The label “IDT” indicate idle times when no task is available for execution. Using IAP-FP scheduling approach, task of all cores are scheduled in low criticality mode as shown in Figure 8, Figure 9, Figure 10 and Figure 11.

Figure 8 shows the execution of four tasks (τ_2 , τ_8 , τ_4 , and τ_6) of group G1 on core C_1 , over a timeline from 0 to 40 units

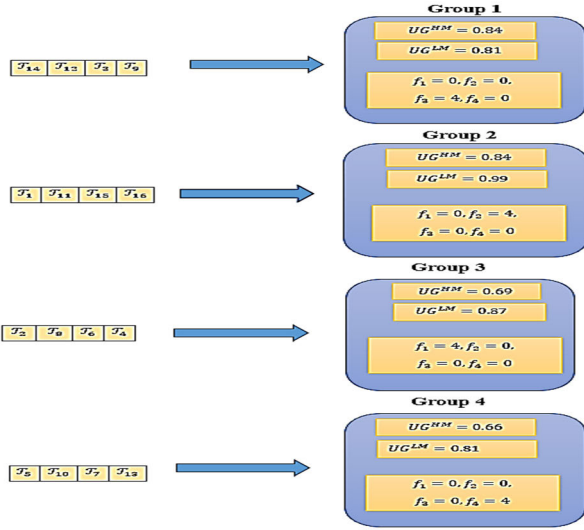
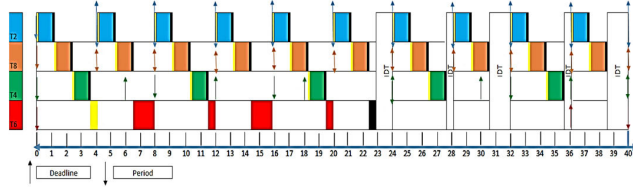
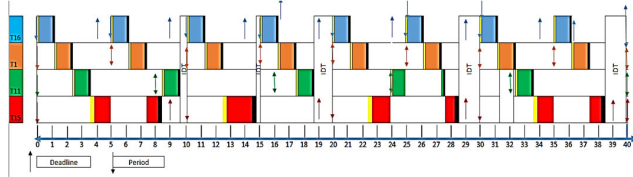
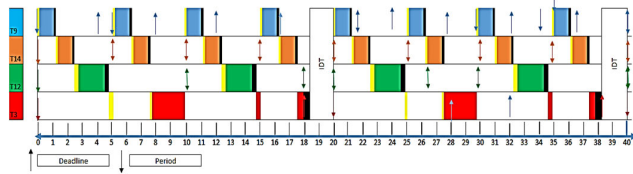
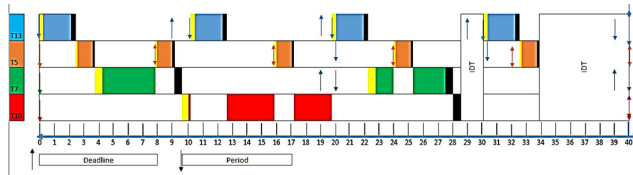


FIGURE 7. Tasks in groups after swapping.

FIGURE 8. Scheduling of tasks of G_1 on Core C_1 .FIGURE 9. Scheduling of tasks of G_2 on Core C_2 .FIGURE 10. Scheduling of tasks of G_3 on Core C_3 .FIGURE 11. Scheduling of tasks of G_4 on Core C_4 .

of time in low criticality mode. The illustration clearly indicates that no task has failed to meet its deadline. Figure 9 illustrates that tasks τ_{16} , τ_{11} , and τ_{15} are scheduled on core C_2 in low criticality mode in such a way that they have completed their execution before their respective deadlines,

ensuring the system meets real-time constraints. Figure 10 illustrates the scheduling of τ_9 , τ_{14} , τ_{12} , and τ_3 on core C_3 in a low criticality mode. The illustration reveals that every task is scheduled according to its designated deadline, with no tasks exceeding their time limits. Figure 11 illustrates the scheduling of tasks τ_{13} , τ_5 , τ_7 and τ_{10} across core C_4 while operating in a low criticality mode. The illustration conveys that all tasks have been assigned within their designated deadlines, and no task has exceeded its specified deadline.

C. SCHEDULING OF TASKS IN HIGH MODE

At first, the mixed criticality tasks are scheduled in low criticality mode, utilizing the C_i^{LO} execution time unit for every task. Following the transition from low criticality to high criticality, the cores consist solely of high criticality tasks that are to be executed in high criticality mode, using the C_i^{HI} execution time unit for each high criticality task. Using IAP-FP scheduling approach, task of all cores are scheduled in high criticality mode as shown in Figure 12, Figure 13, Figure 14 and Figure 15.

The Figure 12 shows that high critical tasks τ_4 and τ_6 are scheduled on core C_1 , showing each task completes its execution within the specified deadline. Core C_2 , schedule high critical tasks τ_{16} and τ_{15} of group G_2 . The Figure 13 indicates that both tasks are scheduled successfully. The Figure 14, illustrates that high critical tasks τ_9 and τ_3 of group G_3 are scheduled without missing their deadlines. The Figure 15, displays that high critical tasks τ_{13} and τ_7 of group G_4 are scheduled over core C_4 , without missing their deadlines.

D. SCHEDULING OF TASKS IN MODE CHANGE

If a high criticality task τ_i is executed for C_i^{LO} time units in low mode and does not signal its completion, the system is switched to high criticality mode, and the task requires additional $C_i^{LO} - C_i^{HI}$ execution time units in high mode. The task must complete C_i^{LO} execution time units in its relative deadline RD_i in low mode, such that it can successfully complete the additional execution time units before or at the actual deadline D_i in high mode. Similarly, a high criticality task τ_i that is released before mode switch and executed for x time units (x is less than C_i^{LO} in low mode, requires additional $C_k - x$ execution time units in high mode, for successful completion in its actual deadline D_i^k . After mode switch, a high criticality task τ_i that is released in high criticality mode, requires C_i^{HI} execution time units in high criticality mode, for the successful completion in its actual deadline D_i . The utilizations on different criticality levels are represented by, U_i^{LO} for low criticality level and U_i^{HI} for high criticality level. The mode change is occurred at 10 unit of time. After 10 unit of time all low critical tasks are dropped from each core. The following four diagrams show the scheduling of tasks of four groups over each core, after mode change. Using IAP-FP scheduling approach, task of all cores are scheduled for mode change as shown in Figure 16, Figure 17, Figure 18 and Figure 19. The worst case response

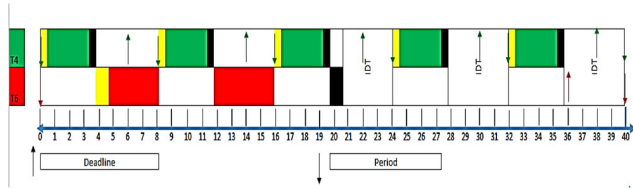


FIGURE 12. Scheduling the high critical tasks of group G_1 over core C_1 .

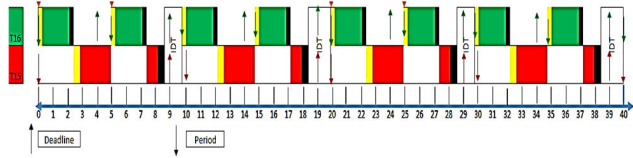


FIGURE 13. Scheduling the high critical tasks of group G_2 over core C_2 .

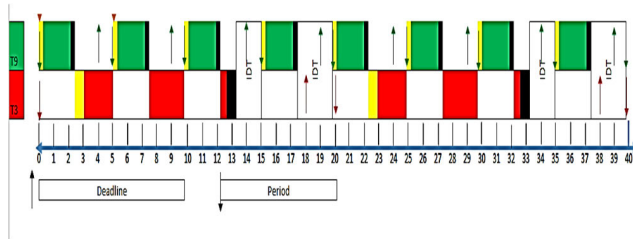


FIGURE 14. Scheduling the high critical tasks of group G_3 over core C_3 .

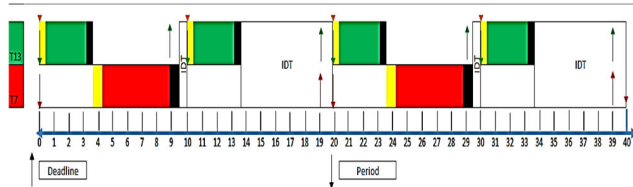


FIGURE 15. Scheduling the high critical tasks of group G_4 over core C_4 .

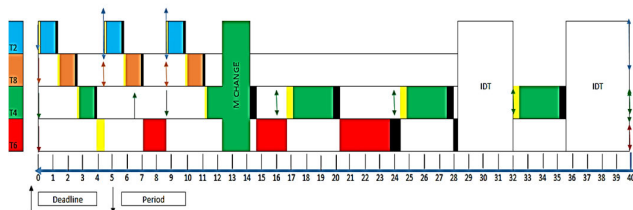


FIGURE 16. Scheduling the tasks of group G_1 over core C_1 in Mode Change.

time for high criticality tasks during mode change $WCRT_i^{MC}$ is calculated using equation 8.

$$WCRT_i^{MC} = \sum_{j \in hp}^n WCRT_j^{HI} + WCRT_j^{LO} + C_i^{HI} + Mint * AP * Tac + Mint * RP * Tac \quad (8)$$

E. WORST CASE RESPONSE TIME ANALYSIS

The Worst Case Response Time $WCRT_i^X$ represents the total time a task takes, including the acquisition phase, computation time, replication phase, and the latency involved in

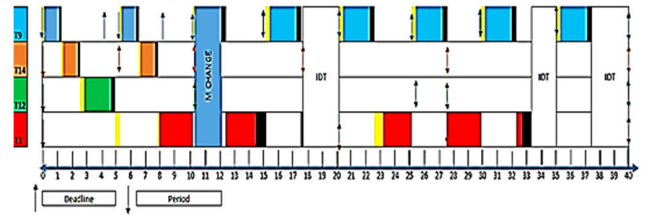


FIGURE 17. Scheduling the tasks of group G_2 over core C_2 in Mode Change.

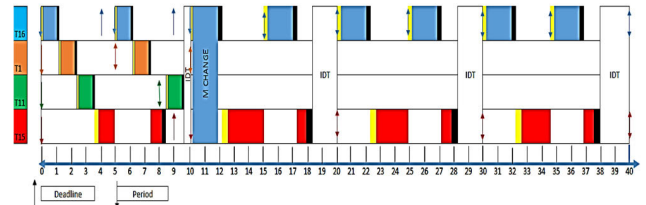


FIGURE 18. Scheduling the tasks of group G_3 over core C_3 in Mode Change.

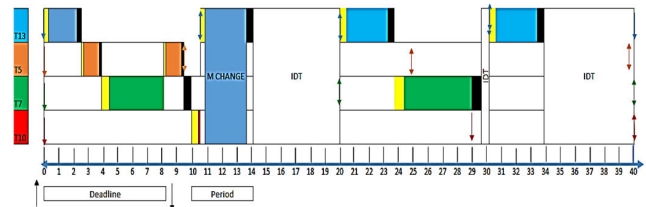


FIGURE 19. Scheduling the tasks of group G_4 over core C_4 in Mode Change.

transitioning between phases. The $WCRT_i^X$ for both low and high mode levels is defined in equation (1) and equation (2) in Section III. Response time analysis is crucial for ensuring tasks are completed within their deadlines. This analysis considers factors like task execution times, priorities, and interference from higher-priority tasks to calculate the maximum response time of a task based on Audsley's priority assignment. The proposed IAP-FP approach for scheduling the mixed-criticality tasks on each core is given in algorithm 8.

The tasks listed in Table 1, which were allocated to cores using the interference-aware, decreasing criticality, and decreasing utilization approach, are now ready to be scheduled. The algorithm 8 illustrates the task scheduling across each core. The first step involves conducting a schedulability test, where the utilization of all tasks is summed up and compared to the total number of cores. If the total utilization exceeds the number of cores, the system cannot be scheduled. However, if the total utilization is less than the number of cores, the system is schedulable. For example, in Example 1, the total utilization of all tasks in low-criticality mode is 3.08375, while the total number of cores is 4, indicating that the system is schedulable in low-criticality mode. Similarly, the total utilization of all high-criticality tasks in

Algorithm 8 IAP-FP Scheduling (Interference Aware Partitioning-Fixed Priority Algorithm)

```

1.  $U^{LO} = \sum_{\tau_i \in T} \{ \frac{C_i(LO)}{P_i} \}$ 
2.  $U^{HI} = \sum_{\tau_i \in T} \{ \frac{C_i(HI)}{P_i} \}$ 
/* Schedulability test */
3. If  $U^{LO} > m$  /*  $m$  is the number of cores */
4. return false /* the system will not schedulable in low mode */
5. If  $U^{HI} > m$ 
6. return false /* the system will not schedulable in high mode */
/* scheduling in low mode. */
7. If  $U^{LO} < m$ 
8. While allocate task  $\tau_n$  and  $\tau_{n+1}$  to the same core
/*assigning interfering tasks  $\tau_n$  and  $\tau_{n+1}$  to the same core*/
9. For each task  $\tau_i \in T$  do /*  $T$  is the task set */
10.  $WCRT_i^{LO} = \sum_{j \in hp}^n WCRT_j^{LO} + C_{ij}^{LO} + \text{Mint} * AP * Tac + \text{Mint} * RP * Tac$ 
11. If  $WCRT_i^{LO} > D_i^{LO}$ 
12. return false
/* scheduling in high mode */
13. If  $U^{HI} < m$ 
14. If  $U^{LO} \leq 0.5$ , then allocate task  $\tau_n$  and  $\tau_{n+1}$  to the same core /*assigning interfering tasks  $\tau_n$  and  $\tau_{n+1}$  to the same core*/
15. For each task  $\tau_i \in T$  do
16.  $WCRT_i^{HI} = \sum_{j \in hp}^{nHI} WCRT_j^{HI} + C_{ij}^{HI} + \text{Mint} * AP * Tac + \text{Mint} * RP * Tac$ 
17. If  $WCRT_i^{HI} > D_i^{HI}$ 
18. return false
/* Schedulability test for every high critical task  $\tau_i$  during mode change for core  $C_k$  */
19. for  $k = k$  to  $n_c$  do
for each HI task  $\tau_i \in T$ 
20.  $WCRT_i^{MC} = \sum_{j \in hp}^n WCRT_j^{HI} + WCRT_j^{LO} + C_{ij}^{HI} + \text{Mint} * AP * Tac + \text{Mint} * RP * Tac$ 
21. if  $WCRT_i^{MC} > D_i^{HI}$  then
22.
23. return false
Exit.
```

high mode is 3.1, which is also less than the total number of cores, making the system schedulable in high mode as well. The next step in our approach is to assign priorities to tasks allocated to the same core. For this, we use Audsley's recursion technique. Using this technique, the priority of each task in low-criticality mode on its assigned core is determined as: $C_1 = \{\tau_2, \tau_1, \tau_5, \tau_6\}$, $C_2 = \{\tau_7, \tau_{10}, \tau_8, \tau_9\}$, $C_3 = \{\tau_3, \tau_4, \tau_{15}, \tau_{16}\}$, and $C_4 = \{\tau_{11}, \tau_{12}, \tau_{13}, \tau_{14}\}$. In the same way priority order of each high critical task on its concern core is: $C_1 = \{\tau_2, \tau_1\}$, $C_2 = \{\tau_7, \tau_9\}$,

TABLE 3. Scheduling of tasks using IAP-FP approach.

Core C_i	T_i	Period $P_i=D_i$	Relative Deadline RD_i	$WCRT_i^{LO}$	$WCRT_i^{HI}$	$WCRT_i^{MC}$
C_1	τ_2	4	4	1.2	-	-
	τ_8	4	4	2.4	-	-
	τ_4	8	6	3.6	3.6	3.4
	τ_6	40	36	8.4	13.2	12.6
C_2	τ_{16}	5	4	1.2	2.4	2.5
	τ_{11}	5	5	2.4	-	-
	τ_{11}	8	8	3.6	-	-
	τ_{15}	10	9	6.0	6.0	6.0
C_3	τ_9	5	4	1.2	2.4	2.5
	τ_{14}	5	5	2.4	-	-
	τ_{12}	10	10	4.8	-	-
	τ_3	20	18	8.4	8.4	8.3
C_4	τ_{13}	10	9	2.4	3.6	3.5
	τ_5	8	8	3.6	-	-
	τ_7	20	19	8.4	9.6	9.4
	τ_{10}	40	40	15.6	-	-

$C_3 = \{\tau_4, \tau_{16}\}$, and $C_4 = \{\tau_{14}, \tau_{13}\}$, these tasks are then executed in this order.

Using Equation 1, we calculate the $WCRT_i^{LO}$ for all tasks in low-criticality mode, and Equation 2 to find the $WCRT_i^{HI}$ for high-criticality tasks in high-criticality mode. A key point to note is the value of mint, which represents the number of cores involved in running tasks that interfere with each other. In the case of Example 1, the value of mint is 1 because the interfering tasks are allocated to the same core. The values of $WCRT_i^X$ for both low and high criticality modes are presented in Table 3. From Table 3, it is evident that the $WCRT_i^X$ for all tasks in both low and high criticality modes is less than their respective deadlines. We can see from the table 3, that our proposed scheduling algorithm takes less scheduling length and also none of the task has missed its deadline.

When the mode changes, low-critical tasks are discarded, and only high-critical tasks are scheduled for execution. Let task τ_2 executes properly, while task τ_4 does not finish its execution in low criticality mode on core C_1 , resulting in mode change. The value of WCRT for task τ_4 during mode change is $WCRT_4^{MC} = 3.4$. The high criticality task τ_{16} and τ_{15} might cause mode change on core C_2 . If τ_{16} causes mode change, then the $WCRT_{16}^{MC} = 2.5$, whereas $WCRT_{15}^{MC} = 6.0$. The tasks τ_3 and τ_9 of core C_3 can trigger mode change. The WCRT of τ_3 and τ_9 for mode change is $WCRT_3^{MC} = 8.3$ and $WCRT_9^{MC} = 2.5$, respectively. Similarly, the WCRT of tasks τ_{13} and τ_{10} on core C_4 for mode change is $WCRT_{13}^{MC} = 3.5$ and $WCRT_{10}^{MC} = 9.4$ respectively. The Table 3 shows values of WCRT for all high critical tasks during mode change.

VII. EXPERIMENTAL EVALUATION

Experimental evaluation in mixed-criticality systems encompasses conducting tests and measurements to assess the

performance and efficiency of various techniques and methodologies. This process allows researchers to validate their proposed solutions and compare them with existing ones. The experimental evaluation shows the effectiveness of our proposed IAP-FP Partitioned method based on IADCDF-WF.

In this research work, the experiments are conducted on randomly generated task sets in a distributed mixed criticality system. The MC task sets are generated using UUnifast algorithm [25]. To demonstrate the efficiency of our interference-aware partitioning approach, we compare it with the Global, MC-Partition [26] and MMC-EDF [27] approaches. The results of the experiments reveal that the IAP-FP method surpasses the other three approaches.

A. TASK SET GENERATION

A set of tasks with different levels of criticality is generated from an initially empty task set. Tasks are randomly added to the ready queue of each processing core to compute the overall utilization. The creation of these random tasks is governed by four adjustable parameters, namely parameter P^{HI} , parameter R^{HI} , parameter m , and parameter n . Parameter P^{HI} , represents the likelihood of high critical tasks appearing in a task set. Parameter R^{HI} defines the maximum ratio between the worst-case computation time in low mode and high mode i.e. $R^{HI} = C_i^{LO}/C_i^{HI}$. Parameter C_i^{LO} represents the worst-case execution time in low mode C_i^{HI} indicates worst-case execution time in high mode. Parameter m is the number of cores used. Lastly, the parameter n signifies the number of task sets. A distributed mixed criticality task is generated randomly as follows:

By using probability P^{HI} , when $X_i = LO$ than the generated task is a low critical task and if $X_i = HI$ than the generated task is a high critical task.

The low mode worst case execution time $C_i(LO)$ is generated within the range of $[1, C_i(LO)^{max}]$, where $C_i(LO)^{max} < C^{max}$.

The high mode worst case execution time $C_i(HI)$ of a task is generated by $C_i(HI) = C_i(LO) \cdot R^{HI}$.

Period of task i.e. P_i is generated within $[1ms, P^{max}]$ range.

A randomly generated task set is created using the distribution of uniform randomness, taking into consideration the desired target utilization $U = U^{LM}$. Generating an exact target utilization can be challenging, thus allowing for U to be generated within a narrow range spanning from $U^{min} + 0.05$ to $U^{max} - 0.05$.

B. RESULTS ANALYSIS

Experiments were conducted through simulations on Intel core i9 system for distributed mixed-criticality task sets using adjustable parameters: P^{HI} values ranging from 0.2 to 1.0, R^{HI} values from 1 to 5, $m = \{4, 6, 8\}$, and $n = \{8, 10, 12, 16\}$. The analysis was performed on 1200 task sets. Figure 20 illustrates the schedulability percentages for various methods, including MC-Partition, Global, MMC-EDF, and the

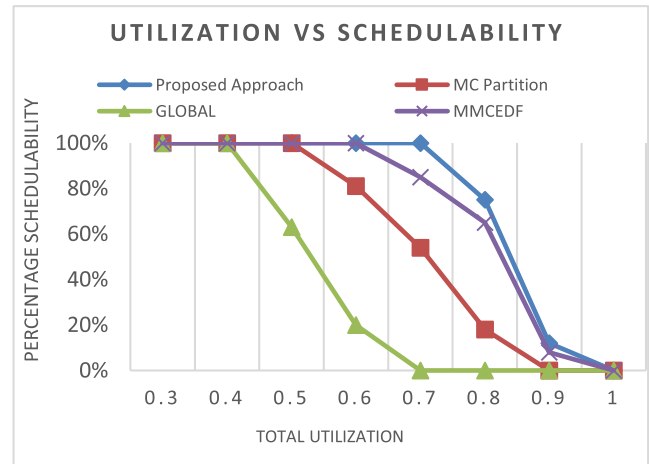


FIGURE 20. Percentage Schedulability VS Net utilization using 4 Cores.

proposed IAP-FP approach. This figure was generated by running random task sets on a four-core system while varying the target utilization from 0.1 to 1.0. At 0.6 utilization, the proposed IAP-FP approach maintains nearly 100% schedulability, while MC-Partition, GLOBAL, and MMCEDF achieve around 81%, 20%, and 100% schedulability, respectively. Similarly, for utilization 0.7, the proposed IAP-FP approach performs 100% schedulability, while MC-Partition, GLOBAL and MMC-EDF schedules 54%, 0%, and 85% of MC task sets respectively. At 0.8 utilization, the Proposed IAP-FP approach continues to outperform the others with 75% schedulability, whereas MC-Partition, GLOBAL, and MMC-EDF further decline to 18%, 0%, and 65% respectively.

The Figure 21 presents a graph illustrating the relationship between the probability of high-criticality tasks and weighted schedulability. Here, probability refers to the likelihood of high-criticality tasks appearing in a task set at a given utilization level. Weighted schedulability simplifies a three-dimensional graph into a two-dimensional representation. This is achieved by assigning weights to various schedulability outcomes based on the challenges associated with scheduling high-utilization task systems. This metric facilitates straightforward comparisons among different scheduling algorithms and highlights intriguing trade-offs between them [25]. The weighted schedulability can be calculated using the formula $AW(u) = \sum U_i e^{U_i} A(U_i) / (\sum U_i) e^{u U_i}$, for a target utilization u , $A(U_i)$ is the acceptance ratio.

For probability of 0.3, the proposed IAP-FP approach shows 85% weighted schedulability while, MC Partition, MMCEDF and GLOBAL show 72%, 82% and 73% weighted schedulability respectively, as shown in Figure 26. Similarly at probability of 0.5, the proposed IAP-FP approach shows 65% weighted schedulability, while MC-Partition, MMCEDF and GLOBAL show 55%, 60% and 50% weighted schedulability. For a probability of 0.7, all approaches show a further decline in terms of weighted schedulability, but the proposed IAP-FP approach still performs better i.e., 53%

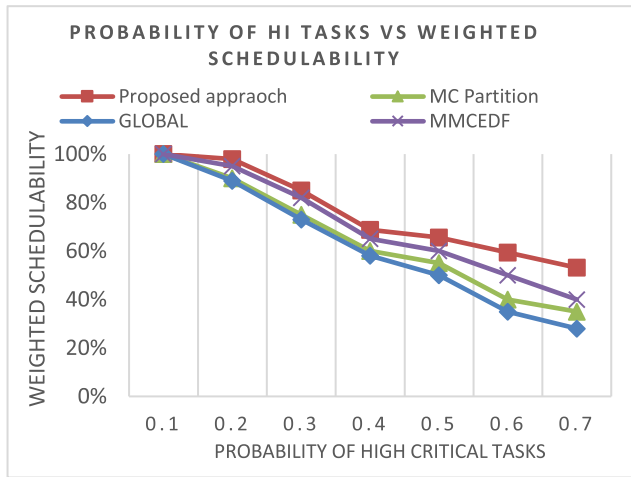


FIGURE 21. Probability of high critical tasks and weighted schedulability for $m = 4$.

weighted schedulability while, MC-Partition, MMCEDF and GLOBAL approaches perform 35%, 40% and 28% weighted schedulability respectively. This detailed analysis highlights the efficiency of the proposed IAP-FP approach in maintaining higher weighted schedulability across varying probabilities of high-critical tasks.

The Figure 22 illustrates the ratios between the worst-case execution times of high-criticality task in low mode and high mode, represented as $(R^{HI} = C_i(LO)^{max}/C_i(HI)^{max})$, for four scheduling approaches: proposed IAP-FP, MC-Partition, Global and MMCEDF. For each R^{HI} , ten acceptance ratios are calculated at various utilization points ranging from 0.1 to 1. The figure shows an inverse relationship between the ratio and weighted schedulability, indicating that as the ratio of low to high criticality task execution times increases, overall system schedulability declines. For a ratio of 1, the proposed IAP-FP approach achieves the highest weighted schedulability i.e., 81%, while, MC-Partition, MMC-EDF and GLOBAL approaches achieves 51%, 70% and 48% weighted schedulability respectively. As the ratio increases to 2, the weighted schedulability of proposed IAP-FP approach decreases to 65% while the weighted schedulability of MC-Partition, MMC-EDF and GLOBAL approaches decrease to 45.5%, 60% and 40% respectively. At a ratio of 3, the proposed IAP-FP approach still performs better and shows 43% schedulability, while MC-Partition drops to 39%, MMC-EDF to 40%, and GLOBAL approach shows at 36% weighted schedulability. This trend highlights the consistent superiority of proposed IAP-FP approach in maintaining higher weighted schedulability across varying ratios of low and high criticality tasks' execution times.

The influence of taskset size on the scheduling feasibility of mixed-criticality tasksets is illustrated in Figure 23. In this graphical depiction, different sets of tasks consisting of 8, 10, 12, and 16 tasks are executed on 4 cores at various utilization levels. At a task set size of 8, the proposed IAP-FP approach leads with a weighted schedulability

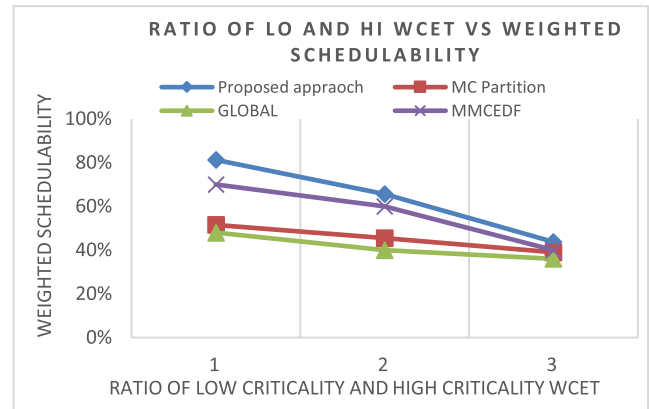


FIGURE 22. Weighted schedulability for the ratio between low WCET and high WCET.

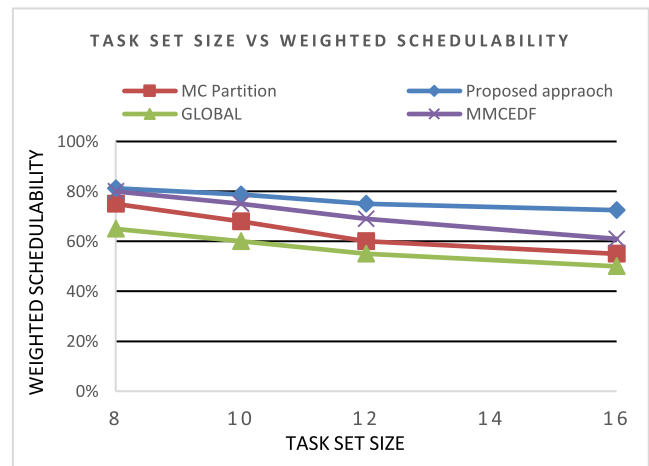


FIGURE 23. Tasks set size and weighted schedulability.

of 81% while, MMC-EDF, MC-Partition and GLOBAL approaches achieve 80%, 75% and 65% weighted schedulability respectively. As the task set size increases to 10, the of proposed IAP-FP approach maintains the highest weighted schedulability at around 80%, while the weighted schedulability of MMC-EDF, MC-Partition and GLOBAL approaches drop to 75%, 68% and 60% respectively. With a task set size of 12, the of proposed IAP-FP approach still outperforms the other approaches and shows 75% weighted schedulability, while MMC-EDF, MC Partition and GLOBAL approaches shows 69%, 60% and 55% weighted schedulability respectively. Similarly, at a task set size of 16, the of proposed IAP-FP approach achieves 72% weighted schedulability while, MMC-EDF, MC-Partition and GLOBAL approaches shows 61%, 55% and 50% weighted schedulability respectively. This detailed analysis demonstrates that the proposed IAP-FP approach performs better in term of weighted schedulability across different task set sizes, highlighting its superior efficiency and effectiveness as compared to the other scheduling methods. The general trend of declining schedulability with increasing task set size is observed across all approaches, but the proposed IAP-FP approach consistently outperforms the others approaches.

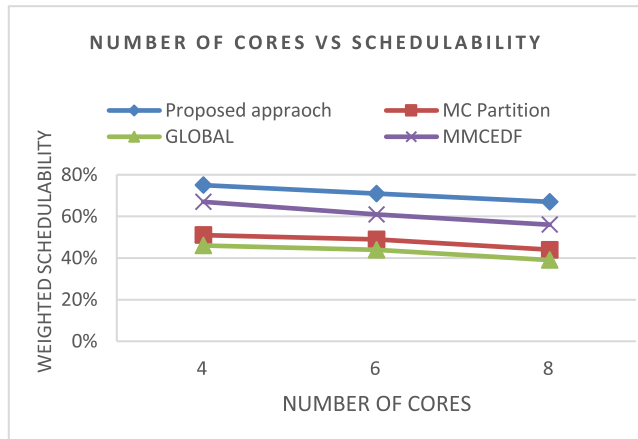


FIGURE 24. Relationship between weighted schedulability and number of cores.

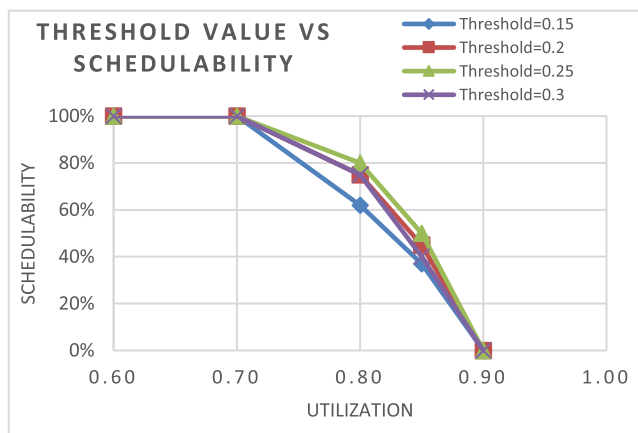


FIGURE 25. Weighted schedulability VS threshold.

The results, shown in Figure 24, reveal an inverse relationship between the numbers of cores at utilization levels ranging from 0.1 to 0.9 and weighted schedulability. For 4 cores, the proposed IAP-FP approach achieves the highest weighted schedulability of 75%, while MMC-EDF, MC-Partition and GLOBAL approaches achieve 67%, 51%, and 46% weighted schedulability respectively. When the number of cores increases to 6, the proposed IAP-FP approach maintains its lead and shows 71% weighted schedulability, while, MMC-EDF, MC-Partition and GLOBAL approaches show 61%, 49%, and 44% weighted schedulability respectively. Similarly, at 8 cores, the proposed IAP-FP approach still performs better and shows 67% weighted schedulability while, MMC-EDF, MC-Partition and GLOBAL approaches achieve 56%, 44% and 39% weighted schedulability respectively. This detailed analysis highlights the superior efficiency of the the proposed IAP-FP approach in maintaining higher weighted schedulability across different numbers of cores, making it the most effective method among the four approaches.

The Figure 25 illustrates the relationship between schedulability and threshold values for the IAP-FP approach, determined through various task sets at different utilizations. Four threshold values ranging from 0.15 to 0.3 are selected,

indicating minimal variance in utilizations among the cores, which led to improved performance for the IAP-FP strategy. At a threshold of 0.15, the IAP-FP approach achieves a 100% schedulability up to utilization $U = 0.7$, but the schedulability declines to 62%, 40%, and 0% at utilizations of 0.8, 0.85, and 0.9, respectively. Increasing the threshold to 0.2 improves schedulability at these higher utilizations, with 75% and 45% schedulability at 0.8 and 0.85, respectively. When the threshold is raised to 0.25, schedulability reaches its peak across all utilization levels; however, a further increase to 0.30 results in a decline in schedulability.

VIII. CONCLUSION AND FUTURE WORK

This research work focuses on enhancing task allocation and scheduling within distributed systems that have varying levels of criticality. In this research work, a novel model for distributed mixed criticality systems, termed the IAP-FP approach, which aims to minimize waiting times among cores caused by memory sharing. To address the challenges associated with memory sharing, a dedicated super model is implemented for memory access. This model is structured into three phases: Access Phase Time (AP), Execution Phase Time (EP), and Replication Phase Time (RP). The IAP-FP approach begins by organizing tasks based on descending criticality and utilization levels, subsequently employing a worst-fit policy for the allocation of interfering tasks to the same cores. For the scheduling of these allocated tasks, Audsley's recursive priority method is applied. Following the prioritization process, a worst-case response time analysis is conducted to effectively schedule the system.

The IAP-FP approach implemented to enhance system performance by improving scheduling length and reducing the deadline miss ratio. Experiments conducted on random task sets demonstrate that the proposed model outperforms the other approaches.

There is potential for our model to be extended to accommodate dependent distributed tasks in future research, which could enhance its applicability in more complex scenarios. This research focuses on the scheduling of tasks within a distributed platform. While the distributed platform encompasses both scheduling and communication, this paper concentrates solely on scheduling. The communication aspect of the distributed platform is beyond the scope of this paper and will be considered for future work involve integrating network communication among the distributed nodes, allowing for improved interaction and data exchange within the system. Additionally, other scheduling methodologies could be investigated and integrated with the dedicated super block models, broadening the model's versatility and effectiveness in various contexts.

LIST OF ACRONYMS

Acronym	Explanation
MC	Mixed Criticality.
WCET	Worst-Case Execution Time.
EDF	Earliest Deadline First.

EDF-VD	EDF With Virtual Deadline.
FpEDF	Fixed-Priority EDF.
EP	Execution Phase.
RP	Replication Phase.
IAP-FP	Interference-Aware Partitioning Fixed Priority.
MMC-EDF	Mixed-Criticality Multicore Compositional <i>Earliest Deadline First</i> .
FTTS	Flexible Time-Triggered Scheduling.
SDG	Sub-block Dependency Graph.
HIB	Height Instruction Count Based.
DSB	Dependent Sub-block Based.
MDS	Maximum Dependency Sub Block.
LLSF	Longest Latency Sub Block First.
MPVD	Mixed Criticality Partitioning with Virtual Deadlines.
AMC	Adaptive Mixed Criticality.
HEFT	Heterogeneous Earliest Finish Time.
CPOP	Critical Path on a Processor.
MH	Mapping Heuristic.
LMT	Level Minimum Time.
DLS	Dynamic Level Scheduling.
TDPATD	Task Deduplication-based Partition Algorithm and Task Duplication.
DMR	Deadline Miss Ratio.
F-HMEFT	Fairness of Multiple Heterogeneous Earliest Finish Time.
D-MHEFT	Deadline Span of Multiple Heterogeneous Earliest Finish Time.
DRAM	Dynamic Random Access Memory.
DCDU	Decreasing Criticality and Decreasing Utilization.
WCRT	Worst-Case Response Time.
IADCDU-WF	Interference-Aware Decreasing Criticality and Decreasing Utilization Worst-Fit.
MEC	Mobile Edge Computing.
PORA	Proactive On-Demand Resource Allocation.
LSTM	Long Short-Term Memory.
BRNN	Bidirectional Recurrent Neural Network.
IoV	Internet of Vehicle.
SARS	Suitability-Based Adaptive Resource Selection.

REFERENCES

- [1] A. Burns and R. I. Davis, "Mixed criticality systems—A review," Tech. Rep., 2022.
- [2] S. Baruah, V. Bonifaci, G. D'Angelo, A. Marchetti-Spaccamela, S. Van Der Ster, and L. Stougie, "Mixed-criticality scheduling of sporadic task systems," in *Proc. Eur. Symp. Algorithms*, Saarbrücken, Germany, Sep. 2011, pp. 555–566.
- [3] G. Giannopoulou, N. Stoimenov, P. Huang, and L. Thiele, "Scheduling of mixed-criticality applications on resource-sharing multicore systems," in *Proc. Int. Conf. Embedded Softw. (EMSOFT)*, Sep. 2013, pp. 1–15.
- [4] S. Osmolovskiy, I. Fedorov, V. Vinogradov, E. Ivanova, and D. Shakurov, "Mixed-criticality scheduling in real-time multiprocessor systems," in *Proc. 18th Conf. Open Innov. Assoc. Seminar Inf. Secur. Protection Inf. Technol. (FRUCT-ISPIIT)*, Apr. 2016, pp. 257–265.
- [5] O. R. Kelly, H. Aydin, and B. Zhao, "On partitioned scheduling of fixed-priority mixed-criticality task sets," in *Proc. IEEE 10th Int. Conf. Trust, Secur. Privacy Comput. Commun.*, Nov. 2011, pp. 1051–1059.
- [6] C. Gu, N. Guan, Q. Deng, and W. Yi, "Partitioned mixed-criticality scheduling on multiprocessor platforms," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2014, pp. 1–6.
- [7] H. Wengang, S. Zhichen, X. Zheng, C. Cen, and K. Li, "A large-scale task scheduling algorithm based on clustering and duplication," *J. Smart Environ. Green Comput.*, vol. 1, no. 4, pp. 202–217, 2021.
- [8] A. Ali and K. H. Kim, "Cluster-based multicore real-time mixed-criticality scheduling," *J. Syst. Archit.*, vol. 79, pp. 45–58, Sep. 2017.
- [9] G. Talmale and U. Shrawankar, "Cluster based real time scheduling for distributed system," *Adv. Distrib. Comput. Artif. Intell. J.*, vol. 10, no. 2, Mar. 2021.
- [10] G. Xie, R. Li, and K. Li, "Distributed computing for functional safety of automotive embedded systems," Tech. Rep., 2012.
- [11] R. M. Pathan, "Schedulability analysis of mixed-criticality systems on multiprocessors," in *Proc. 24th Euromicro Conf. Real-Time Syst.*, Jul. 2012, pp. 309–320.
- [12] D. C. Kiran, S. Gurunaryanan, J. P. Misra, and A. Nawal, "Global scheduling heuristics for multicore architecture," *Sci. Program.*, vol. 2015, no. 1, 2015, Art. no. 860891.
- [13] Z. Al-bayati, Q. Zhao, A. Youssef, H. Zeng, and Z. Gu, "Enhanced partitioned scheduling of mixed-criticality systems on multicore platforms," in *Proc. 20th Asia South Pacific Design Autom. Conf.*, Jan. 2015, pp. 630–635.
- [14] N. Saranya and R. C. Hansdah, "Dynamic partitioning based scheduling of real-time tasks in multicore processors," in *Proc. IEEE 18th Int. Symp. Real-Time Distrib. Comput.*, Apr. 2015, pp. 190–197.
- [15] N. C. Audsley, "Optimal priority assignment and feasibility of static priority tasks with arbitrary start times," Tech. Rep., 1991.
- [16] R. Zakerian and H. Gholami, "SARS: A resource selection algorithm for autonomous driving tasks in heterogeneous mobile edge computing," 2024, *arXiv:2411.15989*.
- [17] H. Gao, X. Wang, W. Wei, A. Al-Dulaimi, and Y. Xu, "Com-DDPG: Task offloading based on multiagent reinforcement learning for information-communication-enhanced mobile edge computing in the Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 348–361, Jan. 2024.
- [18] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [19] G. Xie, G. Zeng, L. Liu, R. Li, and K. Li, "High performance real-time scheduling of multiple mixed-criticality functions in heterogeneous distributed embedded systems," *J. Syst. Archit.*, vol. 70, pp. 3–14, Oct. 2016.
- [20] D. Tamas-Selicean, "Design of mixed-criticality applications on distributed real-time systems," Tech. Univ. Denmark, Lyngby, Denmark, Tech. Rep., 2015.
- [21] R. Jongeling, J. Fredriksson, F. Ciccozzi, A. Cicchetti, and J. Carlson, "Towards consistency checking between a system model and its implementation," in *Proc. Int. Conf. Syst. Modelling Manag.*, Bergen, Norway, Jun. 2020, pp. 30–39.
- [22] A. Dubey, G. Karsai, A. Gokhale, W. Emfinger, and P. Kumar, "DREMS-OS: An operating system for managed distributed real-time embedded systems," in *Proc. 6th Int. Conf. Space Mission Challenges Inf. Technol. (SMC-IT)*, Sep. 2017, pp. 114–119.
- [23] G. Giannopoulou, N. Stoimenov, P. Huang, L. Thiele, and B. D. de Dinechin, "Mixed-criticality scheduling on cluster-based manycores with shared communication and storage resources," *Real-Time Syst.*, vol. 52, no. 4, pp. 399–449, Jul. 2016.
- [24] J. M. Lopez, M. Garc, J. L. Diaz, and D. F. Garcia, "Utilization bounds for multiprocessor rate-monotonic scheduling," *Real-Time Syst.*, vol. 24, pp. 5–28, Jan. 2003.
- [25] E. Bini and G. C. Buttazzo, "Measuring the performance of schedulability tests," *Real-Time Syst.*, vol. 30, nos. 1–2, pp. 129–154, May 2005.
- [26] S. Baruah, B. Chattopadhyay, H. Li, and I. Shin, "Mixed-criticality scheduling on multiprocessors," *Real-Time Syst.*, vol. 50, no. 1, pp. 142–177, Jan. 2014.

- [27] A. Ali, I. Taj, S. Iqbal, M. Fayaz, A. M. Khattak, and B. Hayat, "Mixed criticality multicore compositional framework," *IEEE Access*, vol. 12, pp. 129417–129433, 2024.



AMJAD ALI received the B.Sc. degree in computer science from the University of Peshawar, Pakistan, in 1997, the M.Sc. degree in computer science from Gomal University, Pakistan, in 2000, the M.S. degree in computer science from the University of Peshawar, in 2010, and the Ph.D. degree from Gyeongsang National University, South Korea, in 2016. He was a Lecturer with the University of Peshawar, from 2001 to 2011.

Since 2012, he has been an Assistant Professor with the Department of Computer and Software Technology, University of Swat. His research interests include real-time systems, power-aware computing, and fault-tolerance computing.



SAUD WASLY received the bachelor's degree in electrical and computer engineering from King Abdulaziz University (KAU), in 2007, and the M.Sc. and Ph.D. degrees in electrical and computer engineering from the University of Waterloo, Canada, in 2013 and 2018, respectively. At KAU, he holds multiple leadership roles. He is currently an Assistant Professor with the Department of Electrical and Computer Engineering (Computer Group), KAU. He is the Vice Director of the Center

of Excellence in Intelligent Engineering Systems (CEIES) and the Vice Director of the Industrial Automation Laboratories. He also leads the drone and robotics group within the faculty of engineering. His expertise also encompasses the IoT, robotics, and unmanned systems. He has made significant contributions to national projects, including the national medical ventilator project and the national platform of Wifadah, which serves Hajj pilgrims. His primary research interests include embedded and real-time systems, including system networking and security, computer architecture, operating systems (RTOS), and task scheduling.



ASAD MASOOD KHATTAK (Member, IEEE) received the M.S. degree in information technology from the National University of Sciences and Technology (NUST), Islamabad, Pakistan, in 2008, and the Ph.D. degree in computer engineering from Kyung Hee University, South Korea, in 2012. He joined the College of Technological Innovation, Zayed University, Abu Dhabi, United Arab Emirates, as an Associate Professor, in August 2014. He was a Postdoctoral Fellow

with the Department of Computer Engineering, Kyung Hee University, and an Assistant Professor. He is currently leading three research projects, collaborating in four research projects, and has completed five research projects in the fields of data curation, context-aware computing, the IoT, and secure computing. He has authored/co-authored more than 120 journal and conference articles in highly reputed venues. He is serving as a reviewer, a program committee member, and a guest editor of many conferences and journals. He has delivered keynote speeches, invited talks, guest lectures, and short courses in many universities. He and his team have secured several national and international awards in different competitions.



IHSAN ALI received the Bachelor of Science degree in software engineering from the University of Swat, in December 2015, where he is currently pursuing the Master of Science degree in computer science. His research interests include the domain of distributed mixed-criticality systems, focusing on the design and analysis of systems that efficiently integrate tasks with varying criticality levels in distributed environments.



SHAHID IQBAL received the B.S. and M.S. degrees in computer science from the University of Swat, Swat, Pakistan, in 2016 and 2022, respectively. His research interests include real-time systems, power-aware computing, and fault-tolerance computing.



BASHIR HAYAT received the M.S. degree in computer science from the Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology (SZABIST), Islamabad, Pakistan, in 2010, and the Ph.D. degree in informatics from Gyeongsang National University, Jinju, South Korea, in 2020. He joined the Institute of Management Sciences Peshawar, Pakistan, as a Faculty Member, in 2011.

...